UNIVERSIDAD AUTONOMA DE CHILE

Robots Articulados

Diseña y controla brazos robóticos de 2, 3 y 4 grados de libertad

Caja de herramientas para diseñar tus propios robots: tipos de robots articulados, grados de libertad y sistemas de referencia, cinemática directa e inversa, selección de servomotores y motores paso a paso, y control desde Python por puerto serial.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería

Primera edición · 2026

Robots Articulados
Diseña y controla brazos robóticos de 2, 3 y 4 grados de libertad

Segundo título de la serie de guías educativas. El primero, *Introducción a la Robótica*, construye un robot móvil completo y no es requisito para leer este.

Primera edición, 2026.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería
Universidad Autónoma de Chile

Esta obra se distribuye bajo licencia
Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)

Eres libre de copiar, distribuir y adaptar este material citando la fuente, sin fines comerciales y compartiendo las obras derivadas bajo la misma licencia.

Los archivos del libro (código de cinemática en Python, firmware de Arduino, scripts de control, plantillas y este libro en PDF) están disponibles para descarga libre en el repositorio del libro:
`https://github.com/EduardoFuentesUA/IntroRobotica`

Compuesto con LaTeX (LuaLaTeX, clase KOMA-Script).
Tipografías: TeX Gyre Pagella, TeX Gyre Heros y TeX Gyre Adventor.

El logotipo de la Universidad Autónoma de Chile se reproduce con fines identificatorios. Los nombres de productos y marcas mencionados (Arduino, Python, NEMA, Pololu, Dynamixel) pertenecen a sus respectivos titulares y se citan con fines educativos.

Índice general

0 Prólogo: una caja de herramientas, no un plano 1
- 0.1 Qué es un robot articulado 1
- 0.2 Por qué esta guía no arma un solo robot 2
- 0.3 Las cinco herramientas 2
- 0.4 Qué necesitas antes de empezar 3
- 0.5 Cómo usar este libro 3

1 Robots articulados y sus tipos 4
- 1.1 Anatomía de un robot articulado 4
- 1.2 Los dos tipos de articulación 5
- 1.3 Las seis configuraciones clásicas 5
  - 1.3.1 Cadenas abiertas y cadenas cerradas 7
- 1.4 El espacio de trabajo 7
- 1.5 El brazo de referencia de este libro 7

2 Grados de libertad y sistemas de referencia 10
- 2.1 Qué es un grado de libertad 10
- 2.2 Cuántos grados de libertad pide tu tarea 10
- 2.3 Contar los grados de libertad de un mecanismo 11
- 2.4 Sistemas de referencia 12
- 2.5 Matrices de transformación homogénea 13
- 2.6 Denavit-Hartenberg: cuatro números por eslabón 13
  - 2.6.1 La tabla del brazo de referencia 14
- 2.7 El cero del modelo no es el cero del motor 14

3 Cinemática directa 16
- 3.1 La pregunta y su respuesta 16
- 3.2 Dos grados de libertad en un plano 16
- 3.3 Tres grados de libertad: se agrega la muñeca 17
- 3.4 Cuatro grados de libertad: salir del plano 18
- 3.5 El mismo resultado con Denavit-Hartenberg 19
- 3.6 El código 19
- 3.7 Dibujar el espacio de trabajo 21

4 Cinemática inversa 24
- 4.1 La pregunta difícil 24
- 4.2 Dos grados de libertad: el teorema del coseno 25
  - 4.2.1 Primero el codo 25
  - 4.2.2 Después el hombro 26
- 4.3 Elegir entre codo arriba y codo abajo 27
- 4.4 Tres grados de libertad: el punto de muñeca 27
- 4.5 Cuatro grados de libertad: la base primero 28
- 4.6 Singularidades: dónde el brazo se porta mal 29
- 4.7 Cuando no hay solución cerrada 29
- 4.8 La prueba de ida y vuelta 30

5 Servomotores y motores paso a paso . . . **34**
5.1 Qué le pides a una articulación . . . 34
5.2 El servomotor . . . 35
5.2.1 Cómo se le habla . . . 35
5.2.2 Lo que un servo no hace . . . 35
5.3 El motor paso a paso . . . 36
5.3.1 Micropasos . . . 37
5.3.2 Lo que hay que ajustar sí o sí . . . 37
5.3.3 El origen . . . 37
5.4 Cuál elegir . . . 37
5.5 Dimensionar el par . . . 38
5.5.1 Tres formas de bajar el par que necesitas . . . 39
5.6 Alimentación . . . 40
5.7 Manos a la obra: probar cada articulación por separado . . . 41

6 Control por serial con Python y Arduino . . . **46**
6.1 Repartir el trabajo . . . 46
6.2 Cómo funciona el puerto serial . . . 47
6.3 Diseñar el protocolo . . . 47
6.4 La primera prueba: el eco . . . 47
6.5 Del ángulo del modelo al pulso del servo . . . 49
6.6 El firmware del brazo . . . 50
6.6.1 La interpolación . . . 50
6.6.2 El ángulo que se guarda . . . 51
6.6.3 Los mensajes en memoria de programa . . . 51
6.7 El anfitrión en Python . . . 51
6.8 Manos a la obra: el brazo completo . . . 52
6.9 Depuración . . . 52

7 Proyectos y hacia dónde seguir . . . **54**
7.1 Especificar tu propio robot en seis pasos . . . 54
7.2 Proyectos de 2 grados de libertad . . . 54
7.3 Proyectos de 3 grados de libertad . . . 55
7.4 Proyectos de 4 grados de libertad . . . 55
7.5 La cadena completa: recoger y soltar . . . 56
7.6 Lo que este brazo todavía no hace . . . 57
7.7 Cierre . . . 58

A Formulario . . . **60**
A.1 Notación . . . 60
A.2 Cinemática directa (Capítulo 3) . . . 60
A.3 Cinemática inversa (Capítulo 4) . . . 60
A.4 Denavit-Hartenberg (Capítulo 2) . . . 61
A.5 Movilidad de un mecanismo (Capítulo 2) . . . 61
A.6 Dimensionamiento de actuadores (Capítulo 5) . . . 62
A.7 Calibración de una articulación (Capítulo 6) . . . 62

B Guía de resolución de problemas . . . **63**

C Archivos descargables . . . **65**

0 Prólogo: una caja de herramientas, no un plano

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué distingue a un robot articulado de otros robots.
- Saber por qué esta guía no construye un robot en particular.
- Conocer las cinco herramientas que vas a llevarte, y para qué sirve cada una.
- Saber qué necesitas saber antes de empezar y cómo está organizado el libro.

0.1 Qué es un robot articulado

Un **robot articulado** es una cadena de piezas rígidas unidas por articulaciones motorizadas, con una herramienta en el extremo. Cada articulación aporta un movimiento independiente, y la suma de todos esos movimientos decide dónde termina la herramienta y hacia dónde apunta.

Esa definición cubre desde un brazo industrial de seis ejes hasta una torreta de cámara de dos servos. Cambian el tamaño, el material y el precio; no cambia el problema de fondo, que siempre es el mismo: *dados los ángulos de las articulaciones, ¿dónde está la punta?* y su pregunta inversa, mucho más útil y mucho más difícil, *dado un punto en el espacio, ¿qué ángulos hay que poner?*

La Figura 0.1 compara esas tres escalas sobre una misma superficie de apoyo.

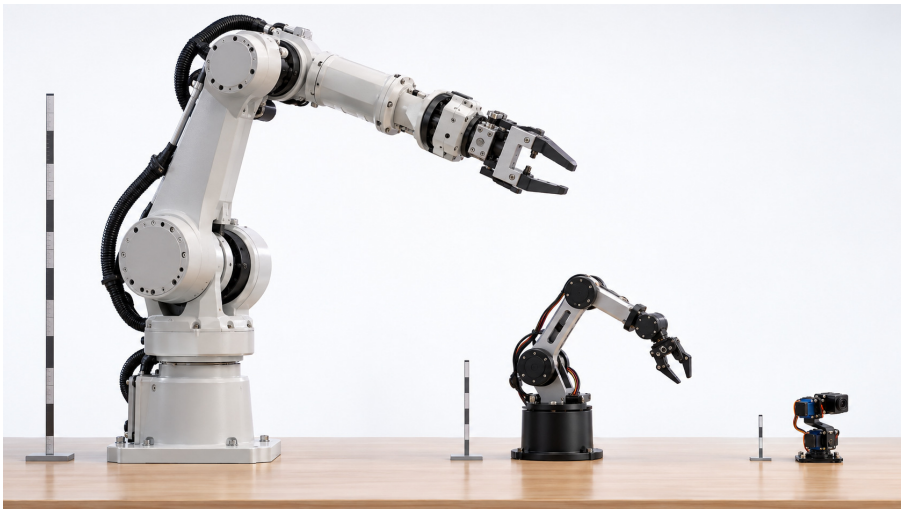


Figura 0.1. Composición ilustrativa de robots articulados a distinta escala: (a) brazo industrial de 6 GDL, (b) brazo de escritorio de 4 GDL y (c) torreta de cámara de 2 GDL.

0.2 Por qué esta guía no arma un solo robot

La guía anterior de esta serie, *Introducción a la Robótica*, construía un robot concreto de principio a fin: un chasis, dos motores y una app. Este libro hace lo contrario, y por una razón deliberada.

Un brazo articulado no tiene una forma correcta. El número de articulaciones, el largo de los eslabones y el tipo de motor dependen de la tarea: un brazo que dibuja sobre una hoja no se parece en nada a uno que traslada piezas de un contenedor a otro, aunque los dos se programen con las mismas ecuaciones. Si el libro te entregara un plano, aprenderías a armar *ese* brazo. Lo que necesitas es poder decidir el tuyo.

Por eso aquí vas a recibir herramientas — con sus fórmulas, su código y sus criterios de decisión — y al final del libro las vas a usar para especificar un robot propio de 2, 3 o 4 grados de libertad.

NOTA

Las herramientas de este libro no dependen de la escala. Las mismas ecuaciones que resuelves para un brazo de escritorio de 250 mm gobiernan un brazo industrial de 2 m; lo único que cambia entre ellos son los actuadores y los números que pones en las fórmulas.

0.3 Las cinco herramientas

El libro entrega cinco cosas, y cada capítulo existe para una de ellas.

Un vocabulario y una taxonomía
: Eslabón, articulación, cadena cinemática, espacio de trabajo. Y la clasificación de los robots articulados, para reconocer qué configuración resuelve tu tarea (Capítulo 1).

Un criterio para elegir cuántos ejes
: Cuántos grados de libertad pide realmente la tarea, ni uno más, y cómo describir la posición de cada pieza con sistemas de referencia (Capítulo 2).

La cinemática
: Directa para saber dónde está la herramienta, inversa para llevarla donde quieres. Con solución cerrada para 2, 3 y 4 grados de libertad, y su código en Python (Capítulos 3 y 4).

Un método para elegir y dimensionar los motores
: Servomotores y motores paso a paso: cuándo conviene cada uno y cómo calcular el par que necesitas antes de comprar nada (Capítulo 5).

Una arquitectura de control
: Python resuelve la geometría, el Arduino mueve los motores, y ambos se hablan por un protocolo serial que tú defines (Capítulo 6).

El Capítulo 7 cierra con un catálogo de proyectos de 2, 3 y 4 grados de libertad, y con el procedimiento para especificar el tuyo.

0.4 Qué necesitas antes de empezar

Este libro asume más que el anterior, pero no mucho más:

- **Trigonometría**: seno, coseno y el teorema del coseno. Nada más avanzado que eso, y las fórmulas se deducen cuando aparecen.
- **Programación básica**: variables, funciones y condicionales. Da lo mismo en qué lenguaje las aprendiste.
- **Electrónica elemental**: qué es un voltaje, una corriente y una masa común. Si vienes del libro anterior, ya la tienes.

No necesitas álgebra lineal previa: las matrices que se usan se explican desde cero y solo se multiplican entre sí.

CONSEJO PRÁCTICO

Todo el código de este libro corre sin hardware. Los capítulos de cinemática se pueden leer, ejecutar y verificar con solo un computador y Python: recién en el Capítulo 5 aparecen los motores. Empieza a leer aunque todavía no hayas comprado nada.

0.5 Cómo usar este libro

Los capítulos 1 a 4 se leen en orden: cada uno usa el vocabulario y las fórmulas del anterior. Los capítulos 5 y 6 son independientes entre sí y puedes tomarlos en cualquier orden, o saltártelos si ya sabes de motores o de comunicación serial.

Vas a encontrar los mismos elementos recurrentes de toda la serie —objetivos al abrir, notas, advertencias, consejos, actividades, verificaciones y resumen al cerrar— más uno propio de este libro: cajas de **ejemplo resuelto**, con un cálculo numérico completo que puedes repetir con lápiz y calculadora para comprobar que entendiste la fórmula.

RESUMEN DEL CAPÍTULO

- Un robot articulado es una cadena de eslabones y articulaciones motorizadas con una herramienta en el extremo.
- Este libro no construye un robot: entrega las herramientas para que diseñes el tuyo, de 2, 3 o 4 grados de libertad.
- Las cinco herramientas son el vocabulario, el criterio de grados de libertad, la cinemática directa e inversa, el dimensionamiento de motores y la arquitectura de control.
- Necesitas trigonometría, programación básica y electrónica elemental; nada de álgebra lineal previa.
- Los capítulos de cinemática se pueden trabajar completos sin comprar ningún componente.

1 Robots articulados y sus tipos

EN ESTE CAPÍTULO APRENDERÁS

- Nombrar correctamente las partes de un robot articulado.
- Distinguir los dos tipos de articulación que vas a usar y reconocer la notación con que se describen los robots.
- Reconocer las seis configuraciones clásicas y saber qué tarea resuelve bien cada una.
- Entender qué es el espacio de trabajo y por qué condiciona todo el diseño.
- Elegir una configuración para tu propio robot con un criterio, no por parecido.

1.1 Anatomía de un robot articulado

Todo robot articulado se describe con cuatro palabras. Vale la pena fijarlas ahora, porque el resto del libro las usa sin volver a definirlas.

Eslabón
: Cada pieza rígida del robot. No se deforma ni cambia de largo: es la parte que diseñas y fabricas.

Articulación
: La unión motorizada entre dos eslabones. Es la parte que se mueve, y la que lleva un motor.

Cadena cinemática
: La secuencia completa de eslabones y articulaciones, desde la base hasta el extremo.

Efector final
: La herramienta del extremo: una pinza, una ventosa, un lápiz, una cámara. Es la única parte del robot cuya posición le importa a la tarea.

La Figura 1.1 reúne esas cuatro partes en una cadena abierta.

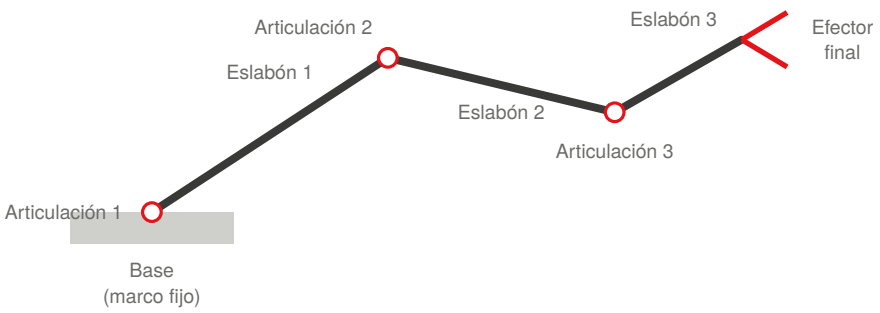


Figura 1.1. Anatomía de una cadena cinemática abierta de tres articulaciones. Los eslabones no cambian de forma; solo se mueven las articulaciones.

NOTA

Cuenta articulaciones, no eslabones. Un robot de tres articulaciones puede tener cuatro o cinco piezas según cómo lo fabriques, pero siempre tendrá tres movimientos independientes, y eso es lo que define lo que puede hacer.

1.2 Los dos tipos de articulación

En la práctica vas a usar solo dos:

- **Rotacional** o de revolución (`R`): los dos eslabones giran uno respecto del otro alrededor de un eje. Es lo que hace un servomotor o un motor paso a paso con su eje directo.
- **Prismática** o lineal (`P`): un eslabón se desliza sobre el otro en línea recta. Es lo que hace un husillo, una correa dentada o un actuador lineal.

Cada una aporta **un** movimiento independiente. Existen articulaciones más complejas —esférica, universal, cilíndrica—, pero casi nunca se motorizan directamente: se construyen combinando articulaciones `R` y `P`.

Un robot se nombra escribiendo su secuencia de articulaciones desde la base hacia la punta. `RRR` es un robot de tres articulaciones rotacionales; `RRP` tiene dos rotacionales y una lineal, en ese orden.

ATENCIÓN

La pinza no es una articulación del robot. Abrir y cerrar una pinza no mueve la herramienta por el espacio: no cuenta como grado de libertad del brazo y no entra en la cinemática. Un brazo «de 4 ejes más pinza» es un robot de 4 grados de libertad, no de 5.

1.3 Las seis configuraciones clásicas

Casi todos los robots industriales caben en seis familias, y todas se distinguen por las primeras tres articulaciones, que son las que posicionan la herramienta.

La Figura 1.2 permite comparar sus articulaciones y sus volúmenes de trabajo; la Tabla 1.1 resume los usos.

De las seis, la **articulada** es la que este libro desarrolla en detalle. Tiene tres ventajas para un primer robot propio: se construye solo con articulaciones rotacionales —o sea, solo con servomotores—, no necesita guías lineales de precisión, y su cinemática tiene solución cerrada sencilla, que es justamente lo que vas a programar en los Capítulos 3 y 4.

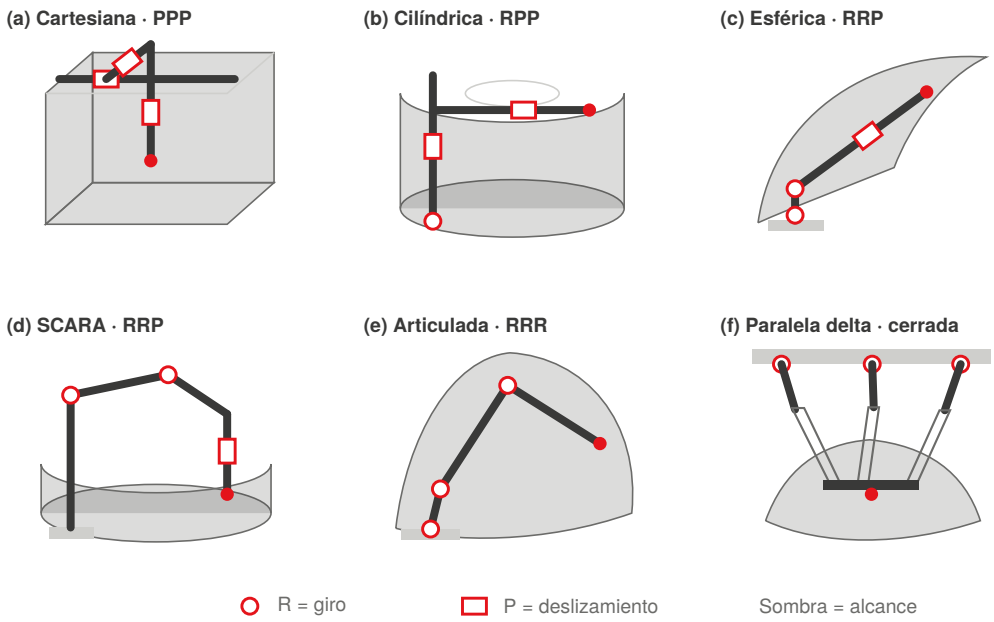


Figura 1.2. Seis configuraciones clásicas: (a) cartesiana, (b) cilíndrica, (c) esférica, (d) SCARA, (e) articulada y (f) paralela delta. La zona sombreada representa el volumen de trabajo de cada familia.

Tabla 1.1. Configuraciones clásicas de robots y su uso típico.

Configuración	Notación	Espacio de trabajo	Dónde se usa y por qué
Cartesiano	PPP	Prisma recto	Impresora 3D, fresadora CNC. Precisión pareja en todo el volumen y programación trivial
Cilíndrico	RPP	Cilindro hueco	Carga y descarga de máquinas. Alcance radial con poca estructura
Esférico o polar	RRP	Casquete esférico	Configuración histórica. Mucho alcance con tres ejes
SCARA	RRP	Cilindro achatado	Ensamblaje electrónico. Rígido en vertical y rápido en el plano
Articulado	RRR	Esfera irregular	Soldadura, pintura, manipulación. Máxima destreza por volumen ocupado
Paralelo (delta)	Cadena cerrada	Casquete achatado	Envasado a alta velocidad. Motores fijos en la base: muy poca inercia

1.3.1 Cadenas abiertas y cadenas cerradas

Las cinco primeras configuraciones son **cadenas abiertas**: una secuencia única de eslabones, cada uno colgando del anterior, como un brazo humano. La sexta, el robot **paralelo**, es una cadena cerrada: varios brazos independientes sujetan la misma plataforma.

La diferencia es práctica. En una cadena abierta cada motor carga con el peso de todo lo que viene después, así que el motor de la base siempre es el más grande. En un robot paralelo los motores viven en la base y los brazos son tubos livianos, lo que permite aceleraciones que una cadena abierta no alcanza —a cambio de un espacio de trabajo mucho más chico y una cinemática bastante más difícil.

NOTA

Todo lo que este libro enseña sobre cinemática vale para cadenas abiertas. Los robots paralelos invierten el problema: en ellos la cinemática *inversa* es la fácil y la directa es la difícil, al revés que en un brazo articulado.

1.4 El espacio de trabajo

El **espacio de trabajo** es el conjunto de puntos que la herramienta puede alcanzar. Es la primera especificación de cualquier robot, antes que el material, el motor o el código: si la pieza que quieres tomar queda fuera, nada de lo demás importa.

Tiene tres características que conviene mirar por separado:

Alcance máximo
: La suma de los largos de los eslabones. Un brazo de 100 y 90 mm nunca llega más allá de 190 mm de su hombro.

Hueco interior
: La zona cercana a la base a la que *tampoco* llega, porque el brazo no puede plegarse más. Con eslabones de 100 y 90 mm, todo lo que esté a menos de 10 mm queda fuera. Es el error más común al ubicar la base del robot: se piensa en el alcance y se olvida el hueco.

Destreza
: La parte del espacio de trabajo donde el robot además puede orientar la herramienta como quieras. Es siempre menor que el espacio alcanzable: en el borde exterior el brazo llega estirado y ya no puede girar la muñeca sin perder el punto.

CONSEJO PRÁCTICO

Diseña el espacio de trabajo con margen. Una regla práctica: que la tarea ocurra entre el 30 % y el 90 % del alcance máximo. Más cerca entras al hueco interior; más lejos trabajas con el brazo casi estirado, donde pierde fuerza y precisión —el Capítulo 4 explica por qué.

1.5 El brazo de referencia de este libro

Para que las fórmulas tengan números concretos, todos los ejemplos usan el mismo brazo articulado de 4 grados de libertad: una base que gira sobre un eje vertical, y sobre ella tres articulaciones de ejes paralelos y horizontales —hombro, codo y muñeca— más una pinza.

Tabla 1.2. Dimensiones del brazo de referencia usado en todos los ejemplos.

Símbolo	Articulación	Valor	Qué mide
d_0	Base (gira en vertical)	60 mm	Altura del suelo al eje del hombro
L_1	Hombro	100 mm	Del eje del hombro al eje del codo
L_2	Codo	90 mm	Del eje del codo al eje de la muñeca
L_3	Muñeca	60 mm	Del eje de la muñeca a la punta de la herramienta

Con esos números, el brazo alcanza como máximo $100+90+60 = 250$ mm medidos desde el eje de la base: una escala de escritorio, que se mueve con servomotores comunes y cabe sobre una mesa. Si tu brazo tiene otras medidas, cambian los números pero no cambia ninguna fórmula.

MANOS A LA OBRA

Antes de seguir al Capítulo 2, define en una hoja el robot que quieres construir:

1. Escribe la tarea en una frase: qué objeto mueve, desde dónde y hasta dónde.
2. Dibuja en planta el área de trabajo con medidas reales, y marca dónde irá la base del robot.
3. Mide la distancia desde la base hasta el punto más lejano y hasta el más cercano de la tarea.
4. Elige una configuración de la Tabla 1.1 y escribe en una línea por qué esa y no otra.

Esta hoja es el punto de partida de todos los capítulos que siguen.

ANTES DE SEGUIR, VERIFICA

- ☐ Puedes nombrar eslabón, articulación, cadena cinemática y efector final señalándolos en la Figura 1.1.
- ☐ Sabes escribir la notación de tu robot (`RRR`, `RRP`…).
- ☐ Elegiste una configuración y sabes justificar la elección.
- ☐ Tienes escritas la distancia máxima y la mínima que tu robot debe alcanzar.

RESUMEN DEL CAPÍTULO

- Un robot articulado es una cadena de eslabones rígidos unidos por articulaciones; solo importa dónde termina el efector final.
- Las articulaciones útiles son dos: rotacional (`R`) y prismática (`P`), y cada una aporta un movimiento independiente.
- La pinza no es un grado de libertad del brazo.
- Las seis configuraciones clásicas resuelven tareas distintas; la articulada (`RRR`) es la

mejor opción para un primer robot propio.
- El espacio de trabajo tiene alcance máximo, hueco interior y zona de destreza: los tres se definen antes de diseñar nada.

2 Grados de libertad y sistemas de referencia

EN ESTE CAPÍTULO APRENDERÁS

- Definir qué es un grado de libertad y cuántos tiene un cuerpo en el plano y en el espacio.
- Decidir cuántos grados de libertad pide tu tarea, ni uno más.
- Contar los grados de libertad de un mecanismo con la fórmula de Grübler.
- Describir la posición y la orientación de un eslabón con matrices de transformación homogénea.
- Armar la tabla de Denavit-Hartenberg de tu robot.

2.1 Qué es un grado de libertad

Un **grado de libertad** (GDL) es un número que hace falta para decir dónde está algo. Ni más ni menos que eso.

Un punto sobre una recta tiene 1: basta con decir a qué distancia está del origen. Un cuerpo rígido sobre una mesa tiene 3: dos para ubicarlo (x, y) y uno para saber hacia dónde está girado (φ). Un cuerpo rígido en el espacio tiene 6: tres de posición (x, y, z) y tres de orientación.

Ese número 6 es el techo del problema. Ningún robot necesita más de seis grados de libertad para poner una herramienta donde sea, apuntando hacia donde sea. Los robots industriales de siete ejes no existen para llegar más lejos, sino para llegar al mismo punto por caminos distintos y esquivar obstáculos.

NOTA

Los grados de libertad de un robot son los de sus *articulaciones motorizadas*. Una articulación que gira libre, sin motor, no aporta un grado de libertad controlable: aporta un problema.

2.2 Cuántos grados de libertad pide tu tarea

Esta es la decisión de diseño más importante del libro, y se toma antes de comprar un solo motor. Cada grado de libertad que agregas cuesta un motor, un eslabón, más peso colgando de los motores anteriores y una ecuación más que resolver. Agregar ejes «por si acaso» es la forma más cara de equivocarse.

La regla es simple: **cuenta cuántos números necesita tu tarea**.

Léela al revés, desde la tarea:

Tabla 2.1. Qué compra cada grado de libertad. La columna crítica es la tercera.

GDL	Cadena	Controlas	No controlas
2	RR en un plano	Posición (x, y) dentro de un plano	La orientación: la herramienta llega girada como salga
3	RRR en un plano	Posición (x, y) y orientación φ en el plano	Nada más, pero todo ocurre en un solo plano
3	R + RR	Posición (x, y, z) en el espacio	La orientación de la herramienta
4	R + RRR	Posición (x, y, z) e inclinación de la herramienta	El giro de la herramienta sobre su propio eje
6	RRR + muñeca	Posición y orientación completas	Nada

- **Dibujar sobre una hoja** con el lápiz siempre vertical: la punta se mueve en un plano y la orientación no importa. **2 GDL**.
- **Tomar piezas de una mesa** con una ventosa que apunta siempre hacia abajo: hay que llegar a cualquier (x, y, z), pero la orientación es fija por construcción. **3 GDL**.
- **Tomar piezas con una pinza** y dejarlas inclinadas, o meterlas en un hueco horizontal: hace falta además controlar la inclinación de la herramienta. **4 GDL**.
- **Soldar un cordón sobre una superficie curva**: la herramienta debe llegar a cualquier punto con cualquier ángulo. **6 GDL**, y ya no es un proyecto de escritorio.

CONSEJO PRÁCTICO

Un truco que ahorra un eje entero: si tu herramienta puede apuntar siempre en la misma dirección —hacia abajo, por ejemplo— fíjala mecánicamente en esa dirección en vez de agregar una articulación para orientarla. Muchos brazos de escritorio de 4 GDL resuelven la misma tarea que uno de 3 GDL con una ventosa vertical, y el de 3 es más rápido, más barato y más rígido.

2.3 Contar los grados de libertad de un mecanismo

En una cadena abierta la cuenta es directa: hay tantos grados de libertad como articulaciones motorizadas. En una cadena cerrada —un robot delta, un pantógrafo, un mecanismo de cuatro barras— no, porque los eslabones se restringen entre sí. Para esos casos existe el criterio de **Grübler**:

$$M = 6\,(N - 1 - J) + \sum_{i=1}^{J} f_i \qquad \text{(en el espacio)}$$

$$M = 3\,(N - 1 - J) + \sum_{i=1}^{J} f_i \qquad \text{(en el plano)}$$

donde M es la movilidad —los grados de libertad del mecanismo—, N el número de eslabones *incluida la base*, J el número de articulaciones y f_i los grados de libertad de cada articulación (1 para las rotacionales y prismáticas).

EJEMPLO RESUELTO

Brazo articulado de 3 GDL. Tiene la base más 3 eslabones, o sea $N = 4$; tres articulaciones rotacionales, $J = 3$ y $\sum f_i = 3$:

$$M = 6\,(4 - 1 - 3) + 3 = 0 + 3 = 3 \;\checkmark$$

Mecanismo plano de cuatro barras. Cuatro eslabones contando la base, cuatro articulaciones rotacionales:

$$M = 3\,(4 - 1 - 4) + 4 = -3 + 4 = 1$$

Un solo grado de libertad, aunque tenga cuatro articulaciones: mueves una y las otras tres quedan determinadas. Por eso un pantógrafo se acciona con un solo motor.

2.4 Sistemas de referencia

Decir que la pinza está «a 150 mm» no significa nada hasta decir *desde dónde* y *en qué dirección*. Un **sistema de referencia** o marco es eso: un origen y tres ejes perpendiculares.

Un robot articulado usa al menos dos, y conviene pensarlos siempre juntos:

- El **marco base** $\{0\}$, fijo, con su origen en el centro de la base del robot. Es donde describes la tarea: «la pieza está en $(170, -60, 40)$».
- El **marco de la herramienta** $\{n\}$, solidario al efector final. Se mueve con el robot y su orientación dice hacia dónde apunta la herramienta.

Los ejes se orientan con la **regla de la mano derecha**: x hacia adelante, y hacia la izquierda, z hacia arriba. Es una convención, pero no es negociable: si mezclas convenciones a mitad de un cálculo, aparecen signos cambiados imposibles de encontrar después.

La Figura 2.1 muestra una vista plana del brazo: el eje y se dibuja vertical en el papel; en el modelo espacial del robot, la altura es z.

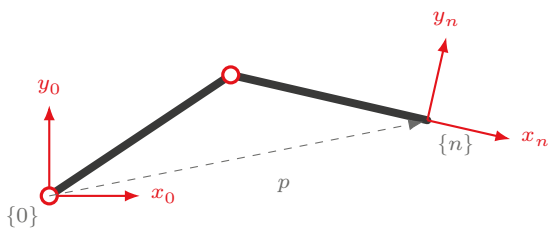


Figura 2.1. En esta vista plana, el marco base $\{0\}$ es fijo y el marco de la herramienta $\{n\}$ viaja con el efector final; el vector p los relaciona.

2.5 Matrices de transformación homogénea

Para saber dónde queda la punta del brazo hay que seguir un marco de referencia por cada eslabón, y de cada marco interesan dos datos: su **posición** (dónde está su origen) y su **orientación** (cuánto está girado respecto del marco anterior). Además, hay que poder pasar de un eslabón al siguiente repitiendo siempre la misma operación.

La **matriz de transformación homogénea** junta la posición y la orientación de un marco en una sola tabla de números. Con ella, pasar de un eslabón al siguiente es multiplicar dos matrices.

En el plano, un giro de ángulo θ seguido de un desplazamiento (p_x, p_y) se escribe:

$$T = \begin{bmatrix} \cos\theta & -\sin\theta & p_x \\ \sin\theta & \cos\theta & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

Las dos primeras columnas guardan la orientación; la tercera, la posición. La última fila no aporta información: está solo para que la matriz sea cuadrada y se pueda multiplicar con otras. En el espacio la idea es idéntica, con una matriz de 4×4:

$$T = \begin{bmatrix} & R & & p \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

donde R es una matriz de rotación de 3×3 y p el vector de posición.

Lo valioso viene ahora. Si ${}^{0}T_{1}$ describe el eslabón 1 visto desde la base, y ${}^{1}T_{2}$ describe el eslabón 2 visto desde el eslabón 1, entonces:

$${}^{0}T_{2} = {}^{0}T_{1}\,{}^{1}T_{2}$$

Encadenar eslabones es multiplicar matrices, en el orden en que van desde la base hacia la punta. Eso es toda la cinemática directa, y es lo que hace el código del Capítulo 3.

ATENCIÓN

El producto de matrices **no es conmutativo**: $A\,B \neq B\,A$. Girar y después desplazar no lleva al mismo lugar que desplazar y después girar. Invertir el orden de dos eslabones en la multiplicación es un error que no produce ningún mensaje de error, solo un robot que se mueve a un punto equivocado.

2.6 Denavit-Hartenberg: cuatro números por eslabón

Escribir a mano la matriz de cada eslabón funciona con dos o tres articulaciones, pero se vuelve inmanejable y propenso a errores. La convención de **Denavit-Hartenberg** (DH) resuelve eso: fija dónde poner los marcos y reduce cada eslabón a cuatro números.

Con esos cuatro números, la matriz del eslabón i queda siempre igual:

Tabla 2.2. Los cuatro parámetros de Denavit-Hartenberg.

Símbolo	Nombre	Qué mide
θ_i	Ángulo de articulación	Giro alrededor de z_{i-1}. *Es la variable* si la articulación es rotacional
d_i	Desplazamiento	Traslación a lo largo de z_{i-1}. Es la variable si la articulación es prismática
a_i	Largo del eslabón	Distancia entre los dos ejes, medida sobre x_i
α_i	Torsión del eslabón	Ángulo entre los ejes z_{i-1} y z_i, medido alrededor de x_i

$$A_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

No hay que memorizarla: se programa una vez —así está en el código del Capítulo 3— y después el trabajo consiste solo en llenar una tabla de cuatro columnas.

2.6.1 La tabla del brazo de referencia

Para el brazo de 4 GDL del Capítulo 1: la base gira sobre un eje vertical y levanta el hombro a la altura d_0; el giro de 90° en α_1 es lo que «acuesta» el eje del hombro para dejarlo horizontal. Las tres articulaciones siguientes tienen ejes paralelos entre sí, así que su torsión es cero.

Tabla 2.3. Tabla DH del brazo de referencia de 4 GDL.

i	Articulación	θ_i	d_i	a_i	α_i
1	Base	θ_0	$d_0 = 60\,\text{mm}$	0	90°
2	Hombro	θ_1	0	$L_1 = 100\,\text{mm}$	0°
3	Codo	θ_2	0	$L_2 = 90\,\text{mm}$	0°
4	Muñeca	θ_3	0	$L_3 = 60\,\text{mm}$	0°

NOTA

Existen dos convenciones DH de uso corriente: la *estándar* —la de esta tabla— y la *modificada* de Craig, que ubica los marcos un eslabón más atrás. Las dos son correctas y llevan al mismo robot, pero producen tablas de parámetros distintas. Elige una y anótala junto a la tabla: mezclar dos libros con convenciones distintas es una fuente clásica de errores.

2.7 El cero del modelo no es el cero del motor

Hay una última referencia, la más olvidada y la que más horas cuesta. Tu modelo matemático define $\theta_1 = 0$ como «el hombro horizontal». Tu servomotor define su propio cero en algún

punto de su recorrido, que no tiene por qué coincidir, y además puede girar en el sentido contrario al que tú consideraste positivo.

Entre ambos hace falta una traducción explícita, con tres datos por articulación: un **desplazamiento** (dónde está el cero del modelo dentro del recorrido del motor), un **signo** (si el motor gira en el mismo sentido que el modelo o al revés) y un **par de límites** (hasta dónde puede moverse sin chocar).

No intentes calcular esos tres números: se miden con el brazo desarmado. El Capítulo 6 trae el procedimiento completo y el firmware que los aplica; por ahora basta con que reserves un lugar para ellos en tu diseño.

MANOS A LA OBRA

Continúa la hoja de diseño que empezaste en el Capítulo 1:

1. Escribe cuántos grados de libertad pide tu tarea, con la frase que lo justifica («necesito controlar x, y, z y la inclinación, entonces 4»).
2. Dibuja el marco base $\{0\}$ sobre tu croquis, con los ejes x, y, z marcados y la regla de la mano derecha respetada.
3. Escribe las coordenadas de los puntos de la tarea en ese marco.
4. Arma la tabla DH de tu robot, con una fila por articulación.

ANTES DE SEGUIR, VERIFICA

- ☐ Puedes decir en una frase cuántos GDL necesita tu tarea y por qué.
- ☐ Tu marco base está dibujado y respeta la regla de la mano derecha.
- ☐ Los puntos de la tarea están escritos como coordenadas en ese marco.
- ☐ Tu tabla DH tiene una fila por articulación y los cuatro parámetros llenos.
- ☐ Sabes que cada articulación necesitará además un desplazamiento, un signo y dos límites, que se miden más adelante.

RESUMEN DEL CAPÍTULO

- Un grado de libertad es un número necesario para describir una posición: 3 en el plano, 6 en el espacio.
- Elegir cuántos GDL es la decisión de diseño más cara del proyecto: cuenta los números que pide la tarea y no agregues ejes de más.
- En cadenas abiertas los GDL son las articulaciones motorizadas; en cadenas cerradas se cuentan con la fórmula de Grübler.
- Una matriz homogénea guarda posición y orientación juntas, y encadenar eslabones es multiplicar matrices en orden.
- Denavit-Hartenberg reduce cada eslabón a cuatro números y convierte el modelo del robot en una tabla.
- El cero del modelo matemático y el cero del motor son cosas distintas y hay que traducir explícitamente entre ellos.

3 Cinemática directa

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué pregunta responde la cinemática directa y por qué siempre tiene una única respuesta.
- Deducir las ecuaciones de un brazo plano de 2 y 3 grados de libertad.
- Extender el resultado a un brazo espacial de 4 grados de libertad.
- Resolver a mano un caso numérico completo y comprobarlo con código.
- Usar la cinemática directa para dibujar el espacio de trabajo de tu robot.

3.1 La pregunta y su respuesta

La **cinemática directa** responde: dados los ángulos de todas las articulaciones, ¿dónde está el efector final y hacia dónde apunta?

Es el problema fácil de los dos, y tiene una propiedad que conviene apreciar: **siempre tiene solución, y es única**. Si pones los motores en unos ángulos determinados, el brazo queda en un solo lugar posible. No hay ambigüedad, no hay casos imposibles, no hay que elegir entre alternativas. Nada de eso será cierto en el capítulo siguiente.

NOTA

Aunque el robot se controla con cinemática inversa, la directa es la que usarás más: para dibujar el espacio de trabajo, para verificar que la inversa no se equivocó, y para simular un movimiento antes de ejecutarlo con motores reales.

3.2 Dos grados de libertad en un plano

Empecemos por el caso mínimo: dos eslabones de largo L_1 y L_2 unidos por articulaciones rotacionales, moviéndose en un plano. Es el brazo de la Figura 3.1.

El truco está en la palabra **relativos**. El ángulo θ_1 se mide desde el eje x, pero θ_2 se mide desde la prolongación del primer eslabón. Entonces, visto desde el eje x, el segundo eslabón apunta en la dirección $\theta_1 + \theta_2$.

Con eso, cada eslabón aporta su proyección horizontal y vertical, y solo hay que sumarlas:

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2) \tag{3.1}$$

$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2) \tag{3.2}$$

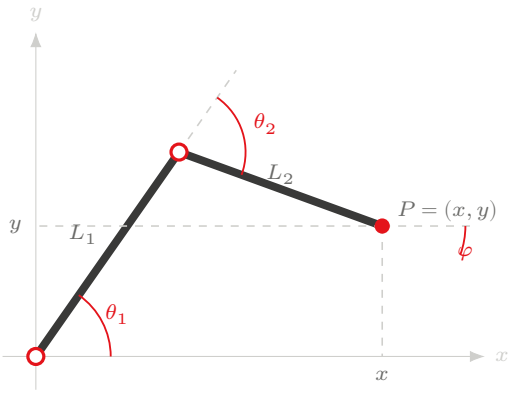


Figura 3.1. Brazo plano de 2 GDL. Los ángulos son *relativos*: θ_2 se mide desde la prolongación del eslabón anterior, no desde el eje x.

Y la orientación del extremo —hacia dónde apunta la herramienta dentro del plano— es la suma de los ángulos:

$$\varphi = \theta_1 + \theta_2 \tag{3.3}$$

CONSEJO PRÁCTICO

Fíjate en que φ no es una variable independiente: queda determinada por θ_1 y θ_2, que ya se usaron para llegar al punto (x, y). Eso es exactamente lo que dice la Tabla 2.1: con 2 GDL controlas la posición y la orientación te sale como salga.

3.3 Tres grados de libertad: se agrega la muñeca

Agregar un tercer eslabón no cambia el método, solo alarga la suma. El tercer eslabón apunta en la dirección $\theta_1 + \theta_2 + \theta_3$:

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3) \tag{3.4}$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3) \tag{3.5}$$
$$\varphi = \theta_1 + \theta_2 + \theta_3 \tag{3.6}$$

La diferencia importante no está en la fórmula sino en lo que ahora puedes hacer: con tres ángulos y tres ecuaciones, puedes fijar x, y y φ a la vez. La muñeca compra el control de la orientación.

El patrón es general: para n eslabones en un plano, cada uno aporta un término con el ángulo acumulado hasta él. Así está escrito el código de este capítulo, con un bucle en vez de tres líneas repetidas.

3.4 Cuatro grados de libertad: salir del plano

Aquí está la idea que hace manejable todo el brazo de 4 GDL, y conviene detenerse en ella porque el Capítulo 4 depende por completo de esto:

> *La base no mueve el brazo dentro de su plano: mueve el plano completo.*

El hombro, el codo y la muñeca tienen ejes paralelos entre sí, así que los tres trabajan siempre sobre un mismo plano vertical. La base gira ese plano alrededor del eje vertical. Entonces el brazo de 4 GDL no es un problema tridimensional: es el problema plano de 3 GDL que ya resolviste, más una rotación al final.

Llamemos r a la distancia horizontal desde el eje de la base hasta la herramienta —la coordenada «hacia afuera» dentro del plano del brazo—:

$$r = L_1\cos\theta_1 + L_2\cos(\theta_1+\theta_2) + L_3\cos(\theta_1+\theta_2+\theta_3) \tag{3.7}$$
$$z = d_0 + L_1\sin\theta_1 + L_2\sin(\theta_1+\theta_2) + L_3\sin(\theta_1+\theta_2+\theta_3) \tag{3.8}$$

y después la base reparte ese r entre x e y:

$$x = r\cos\theta_0 \tag{3.9}$$
$$y = r\sin\theta_0 \tag{3.10}$$
$$\varphi = \theta_1+\theta_2+\theta_3 \tag{3.11}$$

donde φ es ahora la inclinación de la herramienta respecto de la horizontal, medida dentro del plano del brazo.

EJEMPLO RESUELTO

Brazo de referencia ($d_0 = 60$, $L_1 = 100$, $L_2 = 90$, $L_3 = 60$ mm) en la postura $\theta_0 = 30^\circ$, $\theta_1 = 60^\circ$, $\theta_2 = -80^\circ$, $\theta_3 = -40^\circ$.

Paso 1. Ángulos acumulados.

$$\theta_1 = 60^\circ, \qquad \theta_1+\theta_2 = -20^\circ, \qquad \theta_1+\theta_2+\theta_3 = -60^\circ$$

Paso 2. Aporte de cada eslabón.

Eslabón	Horizontal	Vertical
L_1 a 60°	$100\cos 60^\circ = 50.00$	$100\sin 60^\circ = 86.60$
L_2 a -20°	$90\cos(-20^\circ) = 84.57$	$90\sin(-20^\circ) = -30.78$
L_3 a -60°	$60\cos(-60^\circ) = 30.00$	$60\sin(-60^\circ) = -51.96$
Suma	$r = 164.57$	3.86

Paso 3. Altura y giro de la base.

$$z = d_0 + 3.86 = 60 + 3.86 = 63.86 \text{ mm}$$

$$x = 164.57\cos 30^\circ = 142.52 \text{ mm} \qquad y = 164.57\sin 30^\circ = 82.29 \text{ mm}$$

Resultado. La herramienta está en $(142.52,\ 82.29,\ 63.86)$ mm, inclinada $\varphi = -60^\circ$ respecto de la horizontal. Guarda estos números: el Capítulo 4 los usa al revés.

3.5 El mismo resultado con Denavit-Hartenberg

Las fórmulas anteriores son las que vas a programar, porque son directas y rápidas. La tabla DH del Capítulo 2 llega al mismo lugar por otro camino: construye la matriz A_i de cada eslabón y las multiplica en orden.

$$ {}^{0}T_{4} = A_1 \, A_2 \, A_3 \, A_4 $$

La última columna de ${}^{0}T_{4}$ contiene (x, y, z) y el resto de la matriz contiene la orientación completa de la herramienta.

¿Para qué usar el camino largo, entonces? Por dos razones. La primera es que la matriz entrega la orientación completa, no solo el ángulo φ: si tu robot no tiene ejes paralelos, DH es el único camino razonable. La segunda es que sirve de verificación independiente: si las dos formas no coinciden, hay un error en alguna, y encontrarlo es mucho más fácil que descubrirlo con el robot armado.

El código de este capítulo implementa las dos y comprueba que dan el mismo resultado.

3.6 El código

```python
"""Cinematica directa de un brazo articulado de 2, 3 y 4 GDL.

Dos caminos que deben dar el mismo resultado:
  1. Formulas geometricas directas (las que usaras al programar).
  2. Tabla de Denavit-Hartenberg (la que usaras al documentar).

Solo requiere la libreria estandar de Python. Ejecutalo con:
    python cinematica_directa.py
"""

import math

# ---------------------------------------------------------------
# Brazo de referencia del libro (milimetros)
# ---------------------------------------------------------------
D0 = 60.0   # altura de la base al eje del hombro
L1 = 100.0  # hombro -> codo
L2 = 90.0   # codo -> muneca
L3 = 60.0   # muneca -> punta de la herramienta


# ---------------------------------------------------------------
# 1. Camino geometrico
# ---------------------------------------------------------------
def cadena_plana(longitudes, angulos):
    """Extremo de una cadena de eslabones en un plano.

    longitudes: lista de largos, en mm.
    angulos: lista de angulos articulares, en radianes y relativos al
             eslabon anterior (el primero, respecto del eje horizontal).
    Devuelve (x, y, phi): posicion del extremo y orientacion acumulada.
    """
```

```
    x = y = phi = 0.0
    for largo, angulo in zip(longitudes, angulos):
        phi += angulo            # los angulos relativos se suman
        x += largo * math.cos(phi)
        y += largo * math.sin(phi)
    return x, y, phi


def brazo_4gdl(theta0, theta1, theta2, theta3):
    """Cinematica directa del brazo de 4 GDL: base + hombro + codo + muneca.

    Los cuatro angulos van en radianes. Devuelve (x, y, z, phi) en mm y
    radianes, con phi = inclinacion de la herramienta sobre la horizontal.
    """
    # El hombro, el codo y la muneca forman una cadena plana en el plano
    # vertical que la base orienta. r es la distancia al eje de la base.
    r, altura, phi = cadena_plana([L1, L2, L3], [theta1, theta2, theta3])
    x = r * math.cos(theta0)
    y = r * math.sin(theta0)
    z = D0 + altura
    return x, y, z, phi


# ---------------------------------------------------------------
# 2. Camino Denavit-Hartenberg
# ---------------------------------------------------------------
def matriz_dh(theta, d, a, alfa):
    """Matriz homogenea 4x4 de un eslabon, convencion DH estandar."""
    ct, st = math.cos(theta), math.sin(theta)
    ca, sa = math.cos(alfa), math.sin(alfa)
    return [
        [ct, -st * ca,  st * sa, a * ct],
        [st,  ct * ca, -ct * sa, a * st],
        [0.0,      sa,       ca,      d],
        [0.0,     0.0,      0.0,    1.0],
    ]


def multiplicar(A, B):
    """Producto de dos matrices 4x4."""
    return [[sum(A[i][k] * B[k][j] for k in range(4)) for j in range(4)]
            for i in range(4)]


def cinematica_directa_dh(tabla):
    """Compone la tabla DH completa. tabla: lista de (theta, d, a, alfa)."""
    T = [[1.0 if i == j else 0.0 for j in range(4)] for i in range(4)]
    for fila in tabla:
        T = multiplicar(T, matriz_dh(*fila))
    return T


def tabla_dh_brazo(theta0, theta1, theta2, theta3):
    """Tabla DH del brazo de referencia, ya evaluada en una postura."""
    return [
        (theta0, D0, 0.0, math.pi / 2),  # base: gira y levanta el hombro
        (theta1, 0.0,  L1, 0.0),         # hombro
        (theta2, 0.0,  L2, 0.0),         # codo
        (theta3, 0.0,  L3, 0.0),         # muneca
    ]


# ---------------------------------------------------------------
```

```python
# Demostracion
# ---------------------------------------------------------------
if __name__ == "__main__":
    postura = [30.0, 60.0, -80.0, -40.0]          # grados
    t0, t1, t2, t3 = [math.radians(g) for g in postura]

    print("Postura (grados):", postura)

    # --- 2 GDL: solo hombro y codo, en el plano vertical ---
    r2, z2, phi2 = cadena_plana([L1, L2], [t1, t2])
    print("2 GDL  -> r = %7.2f mm   z = %7.2f mm   phi = %6.1f deg"
          % (r2, D0 + z2, math.degrees(phi2)))

    # --- 3 GDL: se agrega la muneca, aparece el control de orientacion ---
    r3, z3, phi3 = cadena_plana([L1, L2, L3], [t1, t2, t3])
    print("3 GDL  -> r = %7.2f mm   z = %7.2f mm   phi = %6.1f deg"
          % (r3, D0 + z3, math.degrees(phi3)))

    # --- 4 GDL: la base saca el brazo del plano ---
    x, y, z, phi = brazo_4gdl(t0, t1, t2, t3)
    print("4 GDL  -> x = %7.2f mm   y = %7.2f mm   z = %7.2f mm   phi = %6.1f deg"
          % (x, y, z, math.degrees(phi)))

    # --- El mismo resultado por Denavit-Hartenberg ---
    T = cinematica_directa_dh(tabla_dh_brazo(t0, t1, t2, t3))
    print("DH     -> x = %7.2f mm   y = %7.2f mm   z = %7.2f mm"
          % (T[0][3], T[1][3], T[2][3]))

    error = max(abs(T[0][3] - x), abs(T[1][3] - y), abs(T[2][3] - z))
    print("Diferencia maxima entre ambos caminos: %.2e mm" % error)
    assert error < 1e-9, "Los dos caminos deberian coincidir"
```

Código 3.1. Cinemática directa por geometría y por Denavit-Hartenberg.

Al ejecutarlo obtienes los mismos números del ejemplo resuelto, por los dos caminos, y la diferencia entre ambos en la última línea:

```
Postura (grados): [30.0, 60.0, -80.0, -40.0]
2 GDL  -> r =  134.57 mm   z =  115.82 mm   phi =  -20.0 deg
3 GDL  -> r =  164.57 mm   z =   63.86 mm   phi =  -60.0 deg
4 GDL  -> x =  142.52 mm   y =   82.29 mm   z =   63.86 mm   phi =  -60.0 deg
DH     -> x =  142.52 mm   y =   82.29 mm   z =   63.86 mm
Diferencia maxima entre ambos caminos: 7.11e-15 mm
```

NOTA

La función `cadena_plana()` no sabe cuántos eslabones tiene el brazo: recibe una lista de largos y una de ángulos. Con la misma función resuelves 2, 3 o 10 grados de libertad, y por eso las tres primeras líneas de la salida salen del mismo código.

3.7 Dibujar el espacio de trabajo

Aquí está el uso más valioso de la cinemática directa. En el Capítulo 1 definiste el espacio de trabajo con dos números —alcance máximo y hueco interior—, pero el espacio real de tu robot no es un anillo perfecto: los límites mecánicos de cada articulación le recortan pedazos.

La forma de averiguar la forma real es por fuerza bruta: recorrer todas las combinaciones de ángulos posibles, calcular la posición con cinemática directa y guardar el punto. Miles de puntos dibujan la silueta, como se ve en la Figura 3.2.

MANOS A LA OBRA

Dibuja el espacio de trabajo de tu robot:

1. Escribe los límites reales de cada articulación de tu diseño, en grados (por ejemplo, el hombro entre $0°$ y $110°$).
2. Modifica el código del Capítulo 3 para recorrer esos rangos de a $5°$ y guardar cada par (r, z) en un archivo de texto.
3. Grafica la nube de puntos, con cualquier herramienta que tengas a mano —una planilla de cálculo sirve.
4. Marca sobre el gráfico los puntos de la tarea que definiste en el Capítulo 1. Si alguno cae fuera de la nube, tienes que cambiar el diseño ahora y no después de comprar los motores.

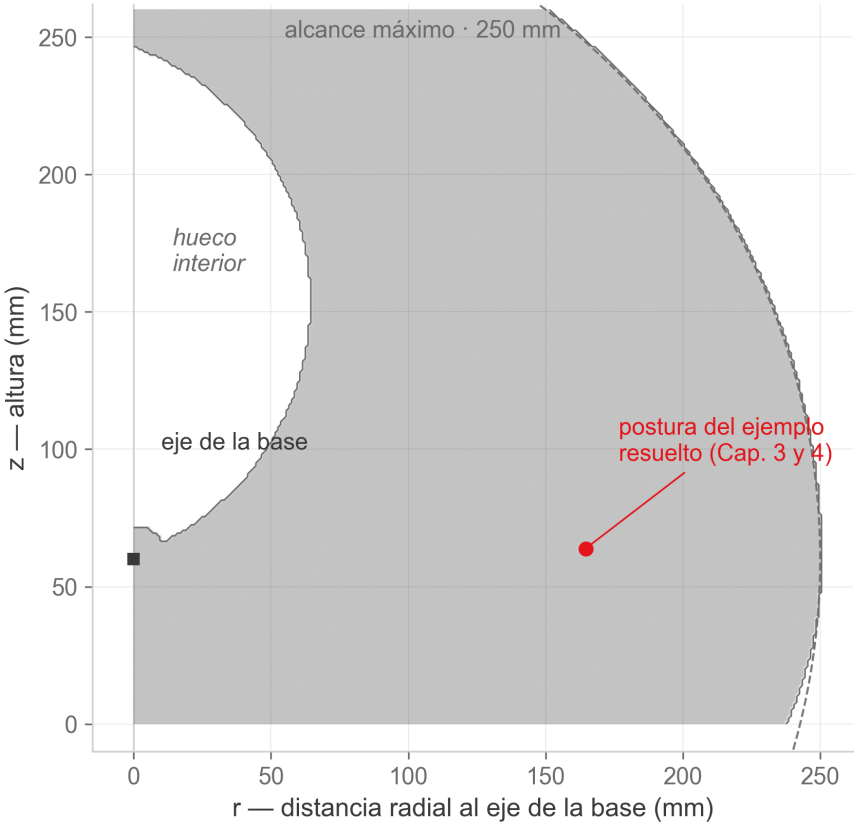


Figura 3.2. Espacio de trabajo del brazo de referencia en el plano (r, z) con límites de ejemplo (hombro 0–$110°$, codo -150–$0°$, muñeca $\pm 100°$). El sombreado muestra el recorte respecto del alcance máximo teórico.

ANTES DE SEGUIR, VERIFICA

- ☐ Reproduje a mano, con calculadora, los números del ejemplo resuelto.
- ☐ El código del capítulo corre en tu computador y muestra la misma salida.

- [ ] La diferencia entre el camino geométrico y el camino DH es del orden de 10^{-15} mm, o sea, cero.
- [ ] Dibujaste el espacio de trabajo de tu propio robot con sus límites reales.
- [ ] Todos los puntos de tu tarea caen dentro de ese espacio.

RESUMEN DEL CAPÍTULO

- La cinemática directa siempre tiene solución y es única: dados los ángulos, hay un solo lugar posible para la herramienta.
- En un brazo plano, cada eslabón aporta su proyección con el ángulo *acumulado* hasta él; los ángulos articulares son relativos al eslabón anterior.
- La orientación en el plano es la suma de todos los ángulos.
- Un brazo de 4 GDL es un brazo plano de 3 GDL más una rotación de la base: resolver r y z primero, repartir después entre x e y.
- Denavit-Hartenberg llega al mismo resultado y sirve como verificación independiente y para robots sin ejes paralelos.
- Barrer los ángulos con cinemática directa es la forma de conocer el espacio de trabajo real, con los límites mecánicos incluidos.

4 Cinemática inversa

EN ESTE CAPÍTULO APRENDERÁS

- Entender por qué la cinemática inversa puede tener cero, una, dos o infinitas soluciones.
- Resolver geométricamente un brazo de 2 GDL con el teorema del coseno.
- Extender la solución a 3 y 4 grados de libertad usando el punto de muñeca.
- Elegir entre soluciones múltiples con un criterio, y detectar los puntos inalcanzables antes de mover un motor.
- Reconocer las singularidades y diseñar el espacio de trabajo lejos de ellas.

4.1 La pregunta difícil

La **cinemática inversa** responde la pregunta que realmente te interesa: dado un punto del espacio, ¿qué ángulos hay que poner en los motores para llegar ahí?

Es el problema difícil, y lo es por una razón concreta: mientras la directa siempre tiene una única respuesta, la inversa puede tener cualquier número de ellas.

Tabla 4.1. Cuántas soluciones tiene la cinemática inversa de un brazo plano de 2 GDL, según dónde esté el punto pedido.

Soluciones	Cuándo	Qué hace tu código
Ninguna	El punto está más lejos que $L_1 + L_2$, o más cerca que $\lvert L_1 - L_2 \rvert$	Avisar, no mover
Una	El punto está justo en el borde: el brazo llega estirado o completamente plegado	Mover, con cuidado
Dos	El punto está dentro del espacio de trabajo: codo arriba y codo abajo	Elegir una
Infinitas	Hay más articulaciones que restricciones (por ejemplo, 3 GDL y solo se pide (x, y))	Fijar la orientación

Cada fila de la tabla le exige algo distinto a tu programa:

- **Ninguna solución** (primera fila). Si el punto pedido queda fuera del alcance del brazo, el programa tiene que detectarlo antes de calcular nada y no mover los motores. Si no lo hace, las fórmulas entregan ángulos sin sentido y esos números llegan igual a los motores.
- **Dos soluciones** (tercera fila). El programa tiene que elegir una con un criterio fijo. Si no lo hace, el brazo puede saltar de una solución a la otra entre dos puntos vecinos y dar un giro brusco que nadie esperaba.

La prueba de alcance se explica en la Sección 4.2, y el criterio para elegir entre las dos soluciones, en la Sección 4.3.

4.2 Dos grados de libertad: el teorema del coseno

El brazo, el punto objetivo y la línea que los une forman un triángulo de lados conocidos: L_1, L_2 y la distancia al objetivo (Figura 4.1). Ahí está toda la solución.

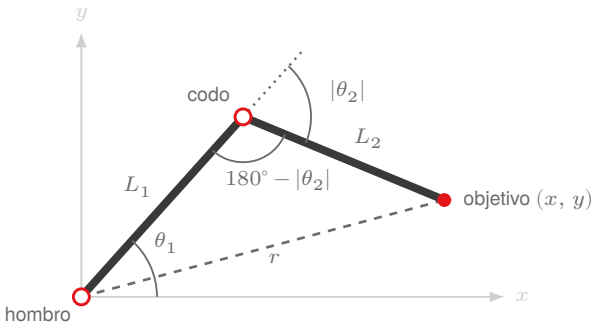


Figura 4.1. El brazo plano de 2 GDL y el punto pedido forman un triángulo de lados L_1, L_2 y r. El ángulo del triángulo en el codo vale $180° - |\theta_2|$, porque θ_2 se mide desde la prolongación del primer eslabón (línea de puntos). En esta postura, codo arriba, θ_2 es negativo: el segundo eslabón gira en sentido horario.

Sea r la distancia desde el hombro hasta el punto (x, y):

$$r = \sqrt{x^2 + y^2} \tag{4.1}$$

4.2.1 Primero el codo

En el triángulo, el ángulo interior en el vértice del codo vale $180° - |\theta_2|$, porque θ_2 se mide desde la prolongación del primer eslabón: cuanto más gira el segundo eslabón respecto de esa prolongación, más se cierra el triángulo. Como el coseno no distingue el signo ($\cos(-\theta_2) = \cos\theta_2$), la ecuación es la misma para las dos soluciones. Aplicando el teorema del coseno a ese triángulo y usando que $\cos(180° - |\theta_2|) = -\cos\theta_2$:

$$r^2 = L_1^2 + L_2^2 + 2L_1L_2\cos\theta_2$$

y despejando:

$$\cos\theta_2 = \frac{r^2 - L_1^2 - L_2^2}{2L_1L_2} \tag{4.2}$$

Esta única ecuación contiene la prueba de alcance y las dos soluciones:

- Si el resultado queda **fuera del intervalo** $[-1, 1]$, no existe ningún triángulo con esos lados: el punto es inalcanzable. Esta comprobación es obligatoria antes de llamar a arccos.

- Si está dentro, arccos entrega un valor, y θ_2 puede ser ese valor *o su negativo*: son las dos soluciones, codo arriba y codo abajo. Con el sentido de giro de este libro (antihorario positivo), un θ_2 **negativo** gira el segundo eslabón en sentido horario respecto de la prolongación del primero y deja el codo **arriba**, como en la Figura 4.1; uno **positivo** deja el codo abajo.

4.2.2 Después el hombro

Con θ_2 conocido, el hombro se calcula como la diferencia de dos ángulos: la dirección hacia el objetivo, menos lo que el codo desvía al brazo de esa dirección.

$$\theta_1 = \operatorname{atan}_2(y,\, x) \;-\; \operatorname{atan}_2(L_2 \sin\theta_2,\; L_1 + L_2\cos\theta_2) \tag{4.3}$$

La función $\operatorname{atan}_2(y, x)$ es el ángulo cuya tangente es y/x, calculado mirando por separado el signo de y y el de x. El 2 forma parte del nombre (se escribe como subíndice para que no se lea como un exponente): indica que la función recibe *dos* argumentos. En los lenguajes de programación se llama `atan2(y, x)`.

ATENCIÓN

Usa siempre `atan2(y, x)` y nunca `atan(y/x)`. La versión de dos argumentos conoce los signos de x e y por separado, así que distingue los cuatro cuadrantes y no divide por cero cuando $x = 0$. Con `atan` simple, la mitad del espacio de trabajo devuelve el ángulo equivocado en $180°$: un brazo que se va exactamente hacia el lado contrario.

EJEMPLO RESUELTO

Alcanzar el punto $(x, y) = (150,\ 40)$ mm con $L_1 = 100$ y $L_2 = 90$ mm, en configuración de codo arriba.

Paso 1. Distancia y prueba de alcance.

$$r^2 = 150^2 + 40^2 = 24\,100 \qquad \Rightarrow \qquad r = 155.24 \text{ mm}$$

Como $|L_1 - L_2| = 10 \le 155.24 \le 190 = L_1 + L_2$, el punto es alcanzable.

Paso 2. Codo.

$$\cos\theta_2 = \frac{24\,100 - 10\,000 - 8\,100}{2 \cdot 100 \cdot 90} = \frac{6\,000}{18\,000} = 0.3333$$

$$\theta_2 = -\arccos(0.3333) = -70.53° \quad \text{(codo arriba; codo abajo sería } +70.53°)$$

Paso 3. Hombro.

$$L_2 \sin\theta_2 = 90 \cdot (-0.9428) = -84.85 \qquad L_1 + L_2\cos\theta_2 = 100 + 30.00 = 130.00$$

$$\theta_1 = \operatorname{atan}_2(40,\ 150) - \operatorname{atan}_2(-84.85,\ 130.00) = 14.93° - (-33.13°) = 48.06°$$

Paso 4. Comprobación con cinemática directa.

$$x = 100\cos 48.06° + 90\cos(-22.46°) = 66.83 + 83.17 = 150.00 \;\checkmark$$

$$y = 100\sin 48.06° + 90\sin(-22.46°) = 74.39 - 34.39 = 40.00 \;\checkmark$$

4.3 Elegir entre codo arriba y codo abajo

La Figura 4.2 compara las dos soluciones para un mismo objetivo.

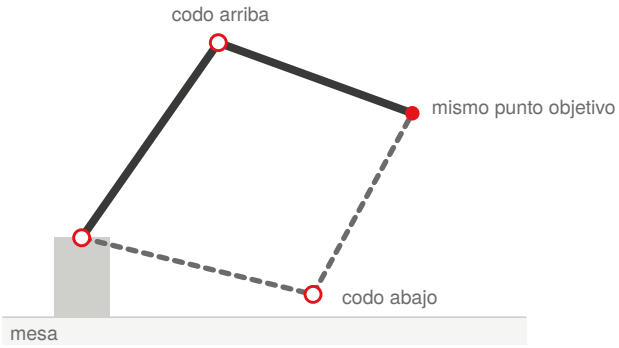


Figura 4.2. Las dos soluciones llevan la herramienta al mismo punto con la misma orientación. Solo una de las dos es utilizable en la práctica.

Las dos soluciones son matemáticamente equivalentes, pero físicamente no. Tres criterios deciden, en este orden:

1. **Límites articulares.** Descarta de entrada cualquier solución cuyos ángulos queden fuera del recorrido mecánico real de tus motores. Suele eliminar una de las dos sin más análisis.
2. **Colisiones.** En un brazo montado sobre una mesa, el codo abajo normalmente hunde el eslabón contra la superficie o contra la base. La convención de casi todos los brazos de escritorio es **codo arriba**.
3. **Continuidad.** Entre dos puntos consecutivos de una trayectoria, elige la solución más parecida a la postura actual. Cambiar de codo arriba a codo abajo a mitad de camino obliga al brazo a un movimiento largo, brusco e imprevisible, aunque los dos puntos estén a milímetros.

CONSEJO PRÁCTICO

Fija la configuración una sola vez, al inicio del programa, y no la cambies durante una trayectoria. Si en algún punto la solución elegida deja de existir, detén el movimiento y avisa: es preferible a que el brazo se dé vuelta solo a mitad de una operación.

4.4 Tres grados de libertad: el punto de muñeca

Con tres articulaciones en un plano puedes pedir posición y orientación: (x, y, φ). La solución no necesita matemática nueva, solo un cambio de punto de vista.

Si sabes que la herramienta debe apuntar en la dirección φ, y sabes que el último eslabón mide L_3, entonces sabes exactamente dónde tiene que estar la muñeca: retrocediendo L_3 desde el objetivo, en la dirección φ.

$$x_m = x - L_3 \cos\varphi \tag{4.4}$$
$$y_m = y - L_3 \sin\varphi \tag{4.5}$$

La Figura 4.3 muestra cómo se obtiene el punto de muñeca retrocediendo desde el objetivo en la dirección φ.

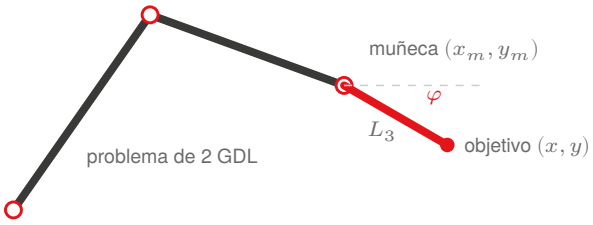


Figura 4.3. Con la orientación φ fijada, el último eslabón queda determinado. Lo que queda por resolver es un brazo de 2 GDL hasta la muñeca.

Ahora resuelves el problema de 2 GDL hasta (x_m, y_m), obtienes θ_1 y θ_2, y el tercer ángulo sale de una resta, porque la orientación es la suma de los tres:

$$\theta_3 = \varphi - \theta_1 - \theta_2 \tag{4.6}$$

NOTA

Fíjate en la consecuencia práctica: la prueba de alcance se hace sobre el *punto de muñeca*, no sobre el objetivo. Un punto que está perfectamente dentro del alcance puede volverse inalcanzable solo por pedir una orientación distinta, porque la muñeca correspondiente queda fuera. Si tu programa avisa «fuera de alcance» en un punto que ves claramente adentro, prueba cambiando φ antes de sospechar del código.

4.5 Cuatro grados de libertad: la base primero

Con el brazo completo el orden es el que ya anticipó el Capítulo 3, y solo tiene tres pasos:

1. **La base apunta al objetivo.** Es lo único que puede hacer:

   $$\theta_0 = \operatorname{atan}_2(y,\, x)$$

   Es la misma función atan_2 de la Sección 4.2.
2. **Se traduce el objetivo al plano del brazo.** La coordenada radial y la altura medida desde el hombro:

   $$r = \sqrt{x^2 + y^2}, \qquad z' = z - d_0$$

3. **Se resuelve el brazo plano de 3 GDL** para el punto (r, z') con la orientación φ pedida.

Eso es todo. El problema espacial de 4 GDL se reduce a una arcotangente y al problema plano que ya sabes resolver.

NOTA

Hay una segunda familia de soluciones: girar la base 180° y alcanzar el punto «hacia atrás», con r negativo. Existe, es válida y algunos robots industriales la usan para esquivar obstáculos. En un brazo de escritorio casi siempre queda fuera de los límites de la base, así que este libro la descarta y trabaja solo con $\theta_0 = \operatorname{atan}_2(y, x)$.

4.6 Singularidades: dónde el brazo se porta mal

Una **singularidad** es una postura donde el brazo pierde la capacidad de moverse en alguna dirección, o donde necesita movimientos articulares enormes para desplazamientos mínimos de la herramienta. No son un error del modelo: son del robot, y por eso se diseñan alrededor de ellas.

Un brazo articulado tiene dos que te van a afectar:

El brazo estirado
: Cuando r se acerca a $L_1 + L_2$, las dos soluciones de codo se juntan en una sola y la sensibilidad se dispara.

El eje de la base
: Cuando el objetivo está sobre el eje vertical de la base, $\operatorname{atan}_2(0, 0)$ no está definido: cualquier θ_0 sirve. Cerca de ese eje, un desplazamiento lateral de milímetros exige que la base gire decenas de grados.

El efecto del primer caso se ve mejor con números que con teoría. Estos son los ángulos del brazo de referencia alcanzando puntos sobre el eje x, con la herramienta horizontal:

Tabla 4.2. Cerca del límite de alcance, el mismo desplazamiento de 20 mm cuesta más del doble de movimiento articular.

x (mm)	θ_1	θ_2	$\Delta\theta_2$ en los últimos 20 mm
120	62.72°	-143.66°	—
140	58.75°	-130.54°	13.1°
220	30.75°	-65.38°	—
240	17.68°	-37.40°	28.0°

En el centro del espacio de trabajo, avanzar 20 mm le cuesta al codo unos 13°. Cerca del borde, los mismos 20 mm le cuestan 28°: el doble de recorrido, el doble de velocidad angular y, con servomotores, un movimiento visiblemente más brusco y menos preciso.

CONSEJO PRÁCTICO

Traduce esto a una regla de diseño: ubica la tarea entre el 30 % y el 90 % del alcance máximo, y evita que pase por encima del eje de la base. Es la misma recomendación del Capítulo 1, ahora con su justificación.

4.7 Cuando no hay solución cerrada

Todo lo anterior funciona porque el brazo tiene ejes paralelos y pocas articulaciones: existe una fórmula exacta. Con seis grados de libertad y ejes en direcciones arbitrarias, esa fórmula puede no existir, y entonces se recurre a métodos **numéricos**: se parte de una postura, se calcula con cinemática directa cuánto falta para el objetivo, se corrige un poco y se repite hasta converger.

Los dos más usados son CCD (*Cyclic Coordinate Descent*, que ajusta una articulación a la vez) y los métodos basados en el jacobiano. Son más lentos, pueden no converger y necesitan una postura inicial razonable.

NOTA

Para 2, 3 y 4 grados de libertad con ejes paralelos —o sea, para todo lo que este libro cubre— siempre existe solución cerrada. Es exacta, se calcula en microsegundos y no puede fallar en converger. No uses métodos numéricos aquí: es cambiar algo que funciona siempre por algo que funciona casi siempre.

4.8 La prueba de ida y vuelta

Existe una forma de verificar la cinemática inversa que no requiere hardware, no requiere confiar en tus cálculos y encuentra prácticamente cualquier error de signo: **resolver la inversa y devolver el resultado a la directa**.

Si los ángulos que entrega la inversa, al pasarlos por la cinemática directa, no devuelven el punto original, hay un error. Si lo devuelven con un error de 10^{-13} mm, tu implementación es correcta.

```
"""Cinematica inversa de un brazo articulado de 2, 3 y 4 GDL.

Solucion cerrada, obtenida por geometria (teorema del coseno). Cada
funcion devuelve None cuando el punto pedido queda fuera del alcance:
la cinematica inversa puede no tener solucion, y el codigo debe decirlo
en vez de entregar un angulo invalido.

Solo requiere la libreria estandar de Python. Ejecutalo con:
    python cinematica_inversa.py
"""

import math

# Brazo de referencia del libro (milimetros)
D0 = 60.0   # altura de la base al eje del hombro
L1 = 100.0  # hombro -> codo
L2 = 90.0   # codo -> muneca
L3 = 60.0   # muneca -> punta de la herramienta


def ik_2gdl(x, y, l1=L1, l2=L2, codo_arriba=True):
    """Dos eslabones en un plano: alcanza el punto (x, y).

    Devuelve (theta1, theta2) en radianes, o None si el punto es
    inalcanzable. El codo tiene dos soluciones: arriba y abajo.
    """
    distancia = math.hypot(x, y)
    if distancia > l1 + l2 or distancia < abs(l1 - l2):
        return None  # fuera del anillo alcanzable

    # Teorema del coseno sobre el triangulo hombro-codo-muneca
    coseno = (distancia ** 2 - l1 ** 2 - l2 ** 2) / (2 * l1 * l2)
    coseno = max(-1.0, min(1.0, coseno))   # protege acos del error de redondeo
    theta2 = -math.acos(coseno) if codo_arriba else math.acos(coseno)

```

```
    # El hombro apunta al objetivo, corregido por lo que desvia el codo
    theta1 = math.atan2(y, x) - math.atan2(l2 * math.sin(theta2),
                                           l1 + l2 * math.cos(theta2))
    return theta1, theta2


def ik_3gdl(x, y, phi, l1=L1, l2=L2, l3=L3, codo_arriba=True):
    """Tres eslabones en un plano: alcanza (x, y) con orientacion phi.

    El tercer grado de libertad se gasta en fijar la orientacion de la
    herramienta, asi que el problema se reduce al de 2 GDL: se retrocede
    l3 desde el objetivo para encontrar la muneca.
    """
    xm = x - l3 * math.cos(phi)
    ym = y - l3 * math.sin(phi)

    solucion = ik_2gdl(xm, ym, l1, l2, codo_arriba)
    if solucion is None:
        return None

    theta1, theta2 = solucion
    theta3 = phi - theta1 - theta2   # lo que falte para cerrar la orientacion
    return theta1, theta2, theta3


def ik_4gdl(x, y, z, phi, codo_arriba=True):
    """Brazo completo: base giratoria mas cadena plana de 3 GDL.

    (x, y, z) en mm desde el centro de la base; phi en radianes, medido
    desde la horizontal. Devuelve los cuatro angulos o None.
    """
    theta0 = math.atan2(y, x)          # la base solo mira hacia el objetivo
    r = math.hypot(x, y)               # distancia al eje de la base
    altura = z - D0                    # alto medido desde el hombro

    solucion = ik_3gdl(r, altura, phi, codo_arriba=codo_arriba)
    if solucion is None:
        return None

    theta1, theta2, theta3 = solucion
    return theta0, theta1, theta2, theta3


def grados(angulos):
    """Convierte una tupla de radianes a grados, redondeada."""
    return tuple(round(math.degrees(a), 2) for a in angulos)


# ---------------------------------------------------------------
# Demostracion: la prueba de ida y vuelta
# ---------------------------------------------------------------
if __name__ == "__main__":
    import sys
    import os
    sys.path.append(os.path.join(os.path.dirname(__file__), "..",
                                 "cap03-cinematica-directa"))
    from cinematica_directa import brazo_4gdl

    objetivo = (142.52, 82.29, 63.86, math.radians(-60.0))
    print("Objetivo: x=%.2f  y=%.2f  z=%.2f  phi=%.1f deg"
          % (objetivo[0], objetivo[1], objetivo[2],
             math.degrees(objetivo[3])))

```

```python
    for nombre, arriba in (("codo arriba", True), ("codo abajo", False)):
        angulos = ik_4gdl(*objetivo, codo_arriba=arriba)
        if angulos is None:
            print("%-12s -> fuera de alcance" % nombre)
            continue

        # Ida y vuelta: la cinematica directa debe devolver el objetivo
        x, y, z, phi = brazo_4gdl(*angulos)
        error = math.dist((x, y, z), objetivo[:3])
        print("%-12s -> angulos %s   error de cierre = %.4f mm"
              % (nombre, grados(angulos), error))

    # Un punto imposible: mas lejos que la suma de los eslabones
    print("\nPunto a 400 mm:", ik_4gdl(400.0, 0.0, 60.0, 0.0))

    # Barrido de alcance sobre el eje X, con la herramienta horizontal
    print("\nAlcance sobre el eje X (phi = 0, codo arriba):")
    for distancia in range(60, 281, 20):
        angulos = ik_4gdl(float(distancia), 0.0, D0, 0.0)
        estado = "ok  %s" % (grados(angulos),) if angulos else "inalcanzable"
        print("  x = %3d mm -> %s" % (distancia, estado))
```

Código 4.1. Cinemática inversa de 2, 3 y 4 GDL, con prueba de ida y vuelta.

Ejecutándolo sobre el mismo punto del ejemplo del Capítulo 3:

```
Objetivo: x=142.52  y=82.29  z=63.86  phi=-60.0 deg
codo arriba  -> angulos (30.0, 60.0, -80.0, -40.0)   error de cierre = 0.0000 mm
codo abajo   -> angulos (30.0, -14.94, 80.0, -125.06)   error de cierre = 0.0000 mm

Punto a 400 mm: None
```

La primera línea devuelve exactamente la postura de la que partimos en el capítulo anterior: la ida y la vuelta cierran. La segunda es la otra solución válida, con el codo abajo. Y el punto a 400 mm, más allá de los 250 mm de alcance, devuelve `None` en vez de un ángulo inventado.

MANOS A LA OBRA

Verifica tu propio brazo antes de construirlo:

1. Cambia d_0, L_1, L_2 y L_3 en el código por las medidas de tu diseño.
2. Pasa por la cinemática inversa los puntos de la tarea que definiste en el Capítulo 1, con la orientación φ que necesites en cada uno.
3. Comprueba que ninguno devuelve `None`, y anota los ángulos que salen.
4. Revisa que esos ángulos caben en el recorrido de los motores que piensas usar. Si alguno pide $200°$, ningún servo común lo va a dar y tienes que rediseñar ahora.

ANTES DE SEGUIR, VERIFICA

- ☐ Reprodujiste a mano el ejemplo resuelto de 2 GDL, incluida la comprobación final.
- ☐ Tu código comprueba el rango de arccos antes de llamarlo y avisa cuando un punto es inalcanzable.
- ☐ Usas `atan2` con dos argumentos en todas partes.
- ☐ Elegiste una configuración de codo y sabes por qué.

- [ ] La prueba de ida y vuelta cierra con error prácticamente nulo para todos los puntos de tu tarea.
- [ ] Ningún punto de la tarea queda a más del 90 % del alcance máximo.

RESUMEN DEL CAPÍTULO

- La cinemática inversa puede tener cero, una, dos o infinitas soluciones; tu código tiene que manejar los cuatro casos.
- El teorema del coseno resuelve el codo, y su condición de existencia es la prueba de alcance: $|L_1 - L_2| \leq r \leq L_1 + L_2$.
- El hombro sale de una diferencia de dos `atan2`: nunca uses `atan` de un solo argumento.
- Con 3 GDL se retrocede L_3 desde el objetivo para hallar la muñeca, y el problema vuelve a ser de 2 GDL.
- Con 4 GDL, la base apunta al objetivo y el resto se resuelve en el plano del brazo.
- Cerca del alcance máximo y sobre el eje de la base el brazo se vuelve impreciso: son singularidades y se evitan por diseño.
- La prueba de ida y vuelta —inversa seguida de directa— detecta casi cualquier error sin necesidad de hardware.

5 Servomotores y motores paso a paso

EN ESTE CAPÍTULO APRENDERÁS

- Saber qué le exige una articulación a su motor, y en qué unidades.
- Entender cómo funcionan y cómo se controlan un servomotor y un motor paso a paso.
- Elegir entre ambos con un criterio técnico.
- Calcular el par que necesita cada articulación de tu brazo *antes* de comprar los motores.
- Alimentarlos sin quemar nada y probar cada articulación por separado.

5.1 Qué le pides a una articulación

Un motor de articulación no se elige por su velocidad ni por sus revoluciones por minuto. Se elige por cinco cosas, en este orden:

Par
: La fuerza de giro que debe sostener contra la gravedad. Es la especificación que decide, y la que se calcula en la Sección 5.5.

Recorrido
: Cuántos grados debe girar. Una articulación de brazo raramente necesita más de 180°; un eje que gira sin fin, sí.

Resolución
: El paso angular más pequeño que puede ordenar. Se paga caro en la punta: un grado en el hombro son varios milímetros en la herramienta.

Repetibilidad
: Si vuelve al mismo lugar cada vez que se lo pides. No es lo mismo que precisión, y para un robot importa más.

Retención
: Si mantiene la posición con la carga colgando, y si la mantiene cuando le cortas la energía.

EJEMPLO RESUELTO

Por qué la resolución importa tanto. El brazo de referencia alcanza 250 mm. Si el hombro se mueve de a 1°, la herramienta salta:

$$250\ \text{mm} \times \frac{\pi}{180} = 4.4\ \text{mm por grado}$$

Cuatro milímetros y medio de salto mínimo. Con la misma articulación mandada en microsegundos —0.09° por microsegundo en un servo típico— el salto baja a 0.4 mm. Es la misma articulación y el mismo motor: cambia solo cómo se le ordena.

5.2 El servomotor

Un servomotor de aficionado —los `SG90`, `MG90S`, `MG996R` y compañía— es una caja que contiene un motor DC, una reductora de engranajes, un potenciómetro que mide el ángulo del eje y un circuito que compara ese ángulo con el que le pides. Si no coinciden, el motor gira hasta que coincidan.

Eso lo convierte en un actuador de **posición absoluta**: le pides $40°$ y va a $40°$, sin importar dónde estaba antes ni si alguien lo movió con la mano. Para un brazo articulado es exactamente lo que quieres, y por eso es el actuador por defecto de este libro.

5.2.1 Cómo se le habla

El servo espera un pulso cada 20 ms (50 Hz). El *ancho* de ese pulso es la orden:

- ≈ 1000 µs: un extremo del recorrido.
- ≈ 1500 µs: el centro.
- ≈ 2000 µs: el otro extremo.

La librería `Servo.h` genera esos pulsos. Tiene dos formas de recibir la orden, y no dan lo mismo:

- `write(grados)` acepta un entero de 0 a 180. Resolución: $1°$.
- `writeMicroseconds(us)` acepta el ancho de pulso directamente. Resolución: unos $0.09°$.

CONSEJO PRÁCTICO

Usa siempre `writeMicroseconds()` en un brazo articulado. Es diez veces más fino, te deja calibrar cada articulación con su propio desplazamiento y su propio signo, y no cuesta nada más. Todo el firmware del Capítulo 6 trabaja en microsegundos.

5.2.2 Lo que un servo no hace

- **No te informa dónde está.** La realimentación es interna. Si la carga lo frena, el servo lo sabe y tú no.
- **No gira sin fin.** Los servos de rotación continua sí lo hacen, pero a cambio pierden el control de posición: el ancho de pulso les indica *velocidad*, no ángulo. No sirven para una articulación de brazo.
- **No es rígido.** Los engranajes tienen holgura, típicamente entre $1°$ y $2°$, que en la punta del brazo de referencia son entre 4 y 9 mm de bamboleo. Es el techo de precisión de un brazo con servos comunes, y ningún código lo arregla.

Tabla 5.1. Servomotores habituales. Los valores son de catálogo y varían entre fabricantes: confirma siempre en la hoja de datos del que compres.

Modelo	Par (kg·cm)	Peso (g)	Pico (A)	Dónde va
`SG90`	1.8	9	0.7	Pinza, muñeca de brazos muy livianos
`MG90S`	2.2	14	0.8	Igual, con engranaje metálico
`MG996R`	9.4	55	2.5	Hombro y codo de brazos de escritorio
`DS3218`	19.0	60	2.5	Hombro con carga útil real

5.3 El motor paso a paso

Un motor paso a paso gira en incrementos fijos. Un `NEMA 17` típico avanza 1.8° por paso, o sea 200 pasos por vuelta. No tiene sensor de posición: funciona en **lazo abierto**, contando pasos desde un origen conocido.

No se conecta al microcontrolador: entre medio va un **driver** (`A4988`, `DRV8825`, `TMC2209`) que recibe dos señales —un pulso por paso en `STEP` y un nivel de sentido en `DIR`— y entrega al motor las corrientes que necesita.

La Figura 5.1 muestra cómo se conecta el conjunto.

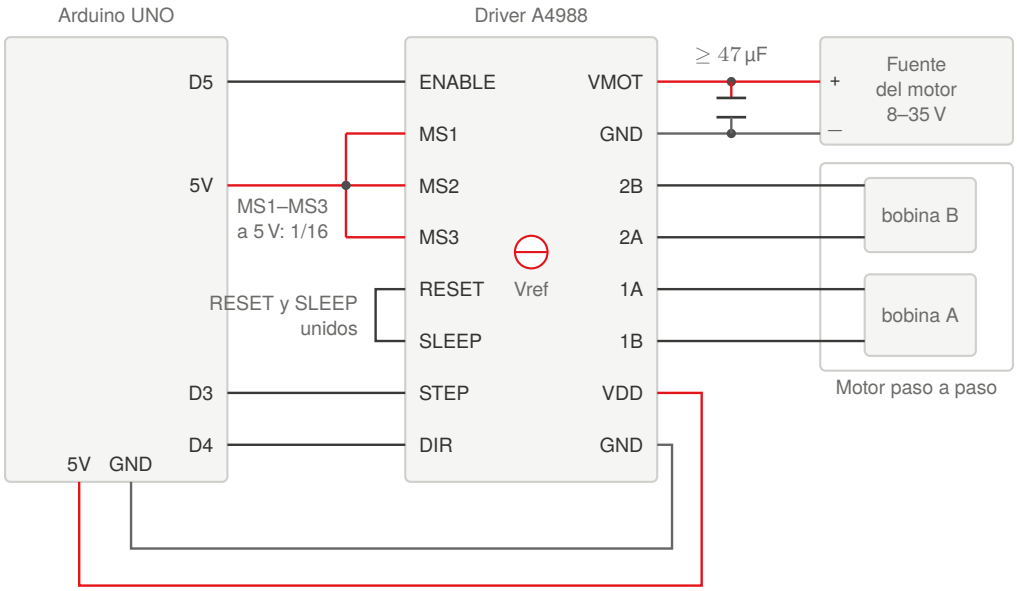


Figura 5.1. Conexión de un motor paso a paso con un driver `A4988`. Cada bobina del motor va a su par de pines (`1A`–`1B` y `2A`–`2B`); `STEP`, `DIR` y `ENABLE` vienen del Arduino, con los pines del Código 5.2. Con `MS1`–`MS3` a 5 V el `A4988` da 1/16 de micropaso (el `DRV8825` usa otra combinación). El capacitor va entre `VMOT` y `GND`, y las masas del motor y del Arduino se unen en el driver. Es un esquema lógico: el orden físico de los pines puede variar entre fabricantes.

5.3.1 Micropasos

El driver puede dividir cada paso en fracciones: 1/2, 1/4, 1/8, 1/16 o más. Con 1/16 de micropaso, el `NEMA 17` pasa de 200 a 3200 posiciones por vuelta, es decir $0.1125°$ por micropaso.

NOTA

El micropaso mejora la suavidad y el ruido mucho más que la precisión real. El par disponible en cada micropaso es menor que el de un paso completo, y la posición efectiva la determinan igual la fricción y la holgura del mecanismo. Úsalo por suavidad, no como si multiplicara la exactitud por dieciséis.

5.3.2 Lo que hay que ajustar sí o sí

ATENCIÓN

Ajusta el límite de corriente del driver antes de conectar el motor. Los `A4988` y `DRV8825` traen un potenciómetro que fija la corriente máxima. De fábrica puede estar muy alto: el driver y el motor se calientan hasta quemarse en minutos. El ajuste se hace midiendo la tensión de referencia (`Vref`) entre el potenciómetro y `GND`, con el motor conectado y detenido, siguiendo la fórmula de la hoja de datos del driver.

ATENCIÓN

Nunca desconectes el motor con el driver energizado. Cortar una bobina en marcha genera un pico de tensión que destruye el driver de inmediato. Corta la alimentación primero, siempre.

5.3.3 El origen

Un motor paso a paso no sabe dónde está al encender: solo sabe cuántos pasos lleva desde que arrancó. Y si la carga lo frena más de lo que su par permite, **pierde pasos en silencio**: el programa cree que el eje avanzó y el eje se quedó atrás. No hay aviso, no hay error, solo un robot desfasado.

De ahí la regla: todo eje con motor paso a paso necesita una **búsqueda de origen** al encender —moverse contra un final de carrera y declarar ese punto como cero— y conviene repetirla cada cierto número de ciclos.

5.4 Cuál elegir

Para un brazo articulado de 2, 3 o 4 GDL, la respuesta casi siempre es **servomotores**: el recorrido es limitado, la reductora ya viene dentro y no necesitas final de carrera en cada eje. Los motores paso a paso ganan en dos casos concretos: una base que debe girar más de $180°$, y un eje prismático movido por husillo o correa, donde el recorrido es largo y la repetibilidad importa.

Tabla 5.2. Servomotor frente a motor paso a paso, en una articulación de brazo.

Criterio	Servomotor	Motor paso a paso
Conexión	Un pin de señal	Dos pines más un driver externo
Posición	Absoluta al encender	Relativa: exige buscar el origen
Recorrido	$\approx 180^\circ$	Ilimitado
Resolución	0.09° en microsegundos	0.1125° con 1/16 de micropaso
Si se sobrecarga	Zumba, se calienta, insiste	Pierde pasos y no avisa
Par por peso	Alto: trae reductora incorporada	Menor, salvo que agregues reducción
Costo	Bajo	Medio (motor más driver)
Cuándo conviene	Articulaciones de brazo de recorrido acotado	Ejes largos, giro continuo, alta repetibilidad

CONSEJO PRÁCTICO

Nada impide mezclarlos. Un diseño frecuente en brazos de escritorio usa un motor paso a paso en la base —recorrido completo y buena repetibilidad— y servos en el hombro, el codo y la muñeca. El firmware del Capítulo 6 se adapta cambiando solo la función que traduce ángulos a órdenes.

5.5 Dimensionar el par

Este es el cálculo que evita comprar dos veces. Se hace con el brazo en su postura más exigente —**completamente horizontal**, que es cuando la gravedad tiene el brazo de palanca más largo— y consiste en sumar momentos.

El par de un servo se especifica en **kilogramo-centímetro**: un servo de 10 kg·cm sostiene 10 kg a 1 cm del eje, o 1 kg a 10 cm. Como la especificación ya es una masa por una distancia, puedes compararla directamente con la suma de tu brazo, sin pasar por newtons:

$$\tau = \sum_i m_i \, d_i \qquad [\text{kg} \cdot \text{cm}] \tag{5.1}$$

donde m_i es cada masa que cuelga más allá de la articulación, en kilogramos, y d_i su distancia horizontal al eje de esa articulación, en centímetros.

EJEMPLO RESUELTO

Par en el hombro del brazo de referencia, con el brazo horizontal y una carga útil de 50 g. Todas las distancias se miden desde el eje del hombro.

Elemento	Masa (kg)	Distancia (cm)	Momento
Eslabón 1 (su centro)	0.040	5.0	0.200
Servo del codo	0.060	10.0	0.600
Eslabón 2 (su centro)	0.030	14.5	0.435
Servo de la muñeca	0.060	19.0	1.140
Pinza	0.040	23.0	0.920
Carga útil	0.050	25.0	1.250
Total			**4.55**

Con factor de seguridad 2: $4.55 \times 2 = 9.1$ kg·cm.
Un `MG996R` (9.4 kg·cm) queda justo en el límite: funcionaría, pero sin margen para aceleraciones ni para que el servo se caliente. Un `DS3218` (19 kg·cm) es la elección correcta para este hombro.

Repitiendo la cuenta para las otras articulaciones, midiendo desde cada eje:

Articulación	Par (kg·cm)	Con FS 2	Servo adecuado
Hombro	4.55	9.1	`DS3218`
Codo	1.95	3.9	`MG996R`
Muñeca	0.46	0.9	`MG90S`
Base	No lucha contra la gravedad: se dimensiona por inercia y fricción		

El hombro necesita *diez veces* el par de la muñeca. Esa es la ley de todo brazo articulado y explica por qué los motores grandes van abajo.
La base no carga el brazo: su eje es vertical y la gravedad no le hace palanca. Lo que le exige el brazo es acelerarlo al girar, y eso se estima con $\tau = J\,\alpha$, donde α es la aceleración angular que quieres y $J = \sum_i m_i\, r_i^2$ es la inercia del brazo completo, con r_i la distancia horizontal de cada masa al eje de la base (en kilogramos y metros, el par sale en N·m). Como r_i va al cuadrado, un brazo estirado pesa mucho más para la base que uno plegado. En la práctica, elige para la base un servo de la misma clase que el del hombro y deja que la interpolación del firmware (Capítulo 6) limite la aceleración.
El factor de seguridad de 2 no cubre movimientos bruscos. La suma de momentos es estática: supone el brazo quieto. Un arranque o un frenado violento con el brazo extendido agrega un par de aceleración que puede superar ese factor, sobre todo en los eslabones largos. Por eso los movimientos de este libro se interpolan en lugar de saltar de una postura a otra.

NOTA

Si necesitas el par en unidades del sistema internacional: $1\ \text{kg}\cdot\text{cm} \approx 0.098\ \text{N}\cdot\text{m}$. El cálculo en kg·cm no es menos riguroso: es el mismo momento, expresado en las unidades en que vienen especificados los servos.

5.5.1 Tres formas de bajar el par que necesitas

Si el número te obliga a un servo demasiado caro, ataca la causa antes de subir de modelo:

1. **Acorta el brazo.** El momento crece linealmente con la distancia, y acortar un eslabón acerca al eje todo lo que viene después: recortar L_2 de 90 a 70 mm baja el par del hombro de 4.55 a

4.22 kg·cm.
2. **Aliviana lo que va lejos.** Cien gramos menos en la pinza le quitan 2.5 kg·cm al hombro; los mismos cien gramos ahorrados en el eslabón 1 le quitan 0.5. Es cinco veces más rentable adelgazar la punta.
3. **Agrega reducción.** Una polea o un engranaje con relación n multiplica el par por n y divide la velocidad y el recorrido por n. A cambio agrega holgura, así que conviene entre 2:1 y 5:1 en articulaciones de brazo.

CONSEJO PRÁCTICO

Un cuarto camino, muy usado en brazos reales: mover el codo desde la base con un mecanismo de paralelogramo. El servo del codo deja de viajar en el extremo del eslabón 1 y pasa a la base, con lo que desaparece de la lista de masas del hombro. En el ejemplo anterior eso solo le quita 0.6 kg·cm al hombro directo, pero en brazos más grandes es la diferencia entre un servo y un motorreductor industrial.

5.6 Alimentación

ATENCIÓN

Nunca alimentes los servos desde el pin de 5 V del Arduino. Ese pin entrega a lo sumo unos cientos de miliamperios; un solo `MG996R` bajo carga pide hasta 2.5 A. El Arduino se reinicia a mitad de un movimiento, o se daña el regulador. Los servos van a una fuente propia de 5–6 V, con la **masa (GND) unida** a la del Arduino.

Para dimensionar esa fuente, suma la corriente *de pico* de todos los motores que puedan moverse a la vez y agrega un margen. Para el brazo de referencia con cuatro servos y una pinza:

$$2.5 + 2.5 + 0.8 + 0.8 \approx 6.6 \text{ A de pico}$$

En un movimiento normal los cuatro servos no llegan al pico a la vez, pero no conviene apostar a eso: elige una fuente de 5–6 V cuya corriente continua sea **igual o mayor que esa suma** (para este brazo, entre 7 y 8 A). Una fuente de 5 A solo sirve si mueves una o dos articulaciones a la vez y con carga liviana.

Además, pon un **capacitor de al menos 1000 µF** entre 5 V y GND, lo más cerca posible de los servos. El capacitor absorbe los picos de arranque, que duran milisegundos: no reemplaza a una fuente corta de corriente sostenida.

La Figura 5.2 muestra cómo se conecta todo: la fuente alimenta a los servos, el Arduino solo les manda la señal, y las masas están unidas.

CONSEJO PRÁCTICO

Un servo que sostiene el brazo horizontal está trabajando casi en condición de bloqueo: consume mucho y se calienta aunque no se mueva. Si tu brazo va a quedar extendido durante minutos, dale una postura de reposo plegada y llévalo ahí cuando no esté trabajando.

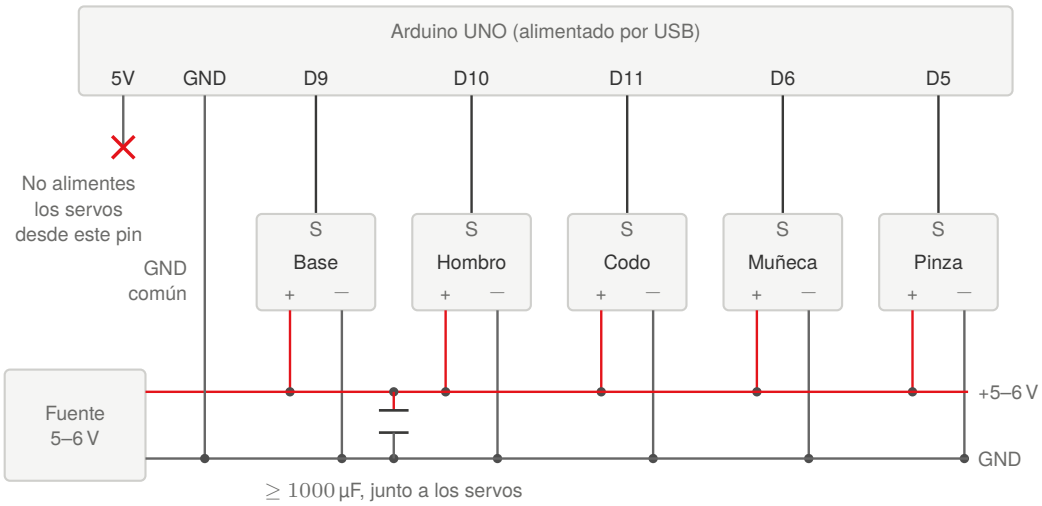


Figura 5.2. Alimentación de los servos del brazo. Los servos toman la energía de una fuente propia de 5–6 V (línea roja), con su masa (gris) unida a la del Arduino, y reciben solo la señal de ángulo del Arduino (trazos oscuros que bajan desde los pines). El capacitor va entre las dos líneas, cerca de los servos. Los pines son los del firmware del Capítulo 6.

5.7 Manos a la obra: probar cada articulación por separado

Antes de montar el brazo, cada articulación se prueba sola. Es media hora que ahorra días: un servo montado que no llega a su ángulo obliga a desarmar todo.

```
// Prueba de una articulación con servomotor: límites por software y
// movimiento por rampa. Este es el sketch mínimo que debes cargar en cada
// articulación nueva ANTES de montarla en el brazo (Capítulo 5).
#include <Servo.h>

Servo articulacion;
const int PIN_SERVO = 9;

// --- Límites mecánicos, en microsegundos de ancho de pulso ---
// Se miden a mano, con el brazo desmontado: mueve el servo de a poco hasta
// que el eslabón toque su tope físico y retrocede 100 us de margen.
const int PULSO_MIN = 900;    // extremo seguro en un sentido
const int PULSO_MAX = 2100;   // extremo seguro en el otro
const int PULSO_HOME = 1500;  // posición de reposo (centro)

// Velocidad del movimiento: cuántos microsegundos avanza por paso.
const int PASO_US = 4;
const int MS_POR_PASO = 12;

int pulsoActual = PULSO_HOME;

// Lleva el servo hasta un pulso objetivo de a poco, no de un salto.
// Un servo sin rampa arranca a máxima velocidad y el brazo rebota.
void moverA(int pulsoObjetivo) {
  pulsoObjetivo = constrain(pulsoObjetivo, PULSO_MIN, PULSO_MAX);
  int paso = (pulsoObjetivo > pulsoActual) ? PASO_US : -PASO_US;

  while (abs(pulsoObjetivo - pulsoActual) > PASO_US) {
    pulsoActual += paso;
    articulacion.writeMicroseconds(pulsoActual);
    delay(MS_POR_PASO);
```

```
  }
  pulsoActual = pulsoObjetivo;
  articulacion.writeMicroseconds(pulsoActual);
}

// Convierte un ángulo del modelo cinemático a ancho de pulso.
// 90 grados de recorrido equivalen a 1000 us en un servo típico.
int anguloAPulso(float grados) {
  return (int)(PULSO_HOME + grados * (1000.0 / 90.0));
}

void setup() {
  Serial.begin(115200);
  articulacion.attach(PIN_SERVO, PULSO_MIN, PULSO_MAX);
  articulacion.writeMicroseconds(PULSO_HOME);
  delay(1000);
  Serial.println(F("Prueba de articulacion lista."));
}

void loop() {
  Serial.println(F("-> +45 grados"));
  moverA(anguloAPulso(45.0));
  delay(800);

  Serial.println(F("-> centro"));
  moverA(PULSO_HOME);
  delay(800);

  Serial.println(F("-> -45 grados"));
  moverA(anguloAPulso(-45.0));
  delay(800);

  Serial.println(F("-> centro"));
  moverA(PULSO_HOME);
  delay(2000);
}
```

Código 5.1. Prueba de una articulación: límites por software y movimiento por rampa.

MANOS A LA OBRA

Para cada articulación de tu brazo, antes de montarla:

1. Conecta el servo solo, sin eslabón, alimentado por su fuente propia y con la masa unida al Arduino.
2. Carga el Código 5.1 y comprueba que se mueve suave, sin saltos ni zumbidos.
3. Monta el eslabón y busca a mano los dos topes mecánicos. Anota el ancho de pulso de cada uno, y réstale 100 µs de margen a cada extremo: esos son `PULSO_MIN` y `PULSO_MAX`.
4. Lleva la articulación a la postura que tu modelo llama «cero grados» y anota el ancho de pulso: ese es `PULSO_CERO`.
5. Manda un pulso mayor y observa si el eslabón gira en el sentido positivo de tu modelo. Si gira al revés, el signo de esa articulación es negativo.
6. Registra los cuatro datos en una tabla. El Capítulo 6 los carga directamente en el firmware.

Si alguna articulación de tu diseño usa un motor paso a paso, el Código 5.2 hace lo equivalente: mueve el eje a ángulos absolutos después de buscar su origen.

```
// Control de un motor paso a paso NEMA 17 con driver A4988 o DRV8825.
// Sin librerías: el driver solo necesita un pulso por paso y un nivel que
```

Tabla 5.3. Hoja de calibración: cuatro datos por articulación, todos medidos.

Articulación	Pulso cero	Signo	Pulso mín.	Pulso máx.
Base	_____	_____	_____	_____
Hombro	_____	_____	_____	_____
Codo	_____	_____	_____	_____
Muñeca	_____	_____	_____	_____

```
// indique el sentido. Incluye búsqueda de origen con final de carrera,
// obligatoria en todo motor paso a paso (Capítulo 5).

const int PIN_PASO = 3;      // STEP del driver
const int PIN_DIR = 4;       // DIR del driver
const int PIN_ENABLE = 5;    // ENABLE del driver (activo en bajo)
const int PIN_FINAL = 6;     // final de carrera del origen, a GND

// --- Geometría del eje ---
const int PASOS_POR_VUELTA = 200;   // NEMA 17 de 1.8 grados por paso
const int MICROPASOS = 16;          // configurado con los jumpers del driver
const float REDUCCION = 5.0;        // relación de la correa o del engranaje

// Micropasos que hay que dar para girar un grado del eslabón
const float PASOS_POR_GRADO =
    (PASOS_POR_VUELTA * MICROPASOS * REDUCCION) / 360.0;

// Ancho del pulso y pausa entre pasos: define la velocidad
const int ANCHO_PULSO_US = 3;
const int PAUSA_PASO_US = 700;

long posicionPasos = 0;   // posición actual, en micropasos desde el origen

void darPaso() {
  digitalWrite(PIN_PASO, HIGH);
  delayMicroseconds(ANCHO_PULSO_US);
  digitalWrite(PIN_PASO, LOW);
  delayMicroseconds(PAUSA_PASO_US);
}

// Gira una cantidad de micropasos. El signo indica el sentido.
void moverPasos(long pasos) {
  digitalWrite(PIN_DIR, pasos >= 0 ? HIGH : LOW);
  long cantidad = abs(pasos);
  for (long i = 0; i < cantidad; i++) {
    darPaso();
  }
  posicionPasos += pasos;
}

// Mueve el eslabón a un ángulo absoluto, medido desde el origen.
void moverAGrados(float grados) {
  long objetivo = (long)(grados * PASOS_POR_GRADO);
  moverPasos(objetivo - posicionPasos);
}

// Un motor paso a paso no sabe dónde está al encender: hay que llevarlo
// contra un final de carrera y declarar ese punto como cero.
void buscarOrigen() {
  Serial.println(F("Buscando origen..."));
  digitalWrite(PIN_DIR, LOW);                // sentido hacia el final de carrera
```

```
  while (digitalRead(PIN_FINAL) == HIGH) {    // HIGH = sin tocar (pull-up interno)
    darPaso();
  }
  moverPasos(20);          // se despega un poco del sensor
  posicionPasos = 0;       // este punto es el cero del eje
  Serial.println(F("Origen encontrado."));
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_PASO, OUTPUT);
  pinMode(PIN_DIR, OUTPUT);
  pinMode(PIN_ENABLE, OUTPUT);
  pinMode(PIN_FINAL, INPUT_PULLUP);

  digitalWrite(PIN_ENABLE, LOW);   // activo: el motor queda con par de retención
  buscarOrigen();
}

void loop() {
  Serial.println(F("-> 30 grados"));
  moverAGrados(30.0);
  delay(1000);

  Serial.println(F("-> 90 grados"));
  moverAGrados(90.0);
  delay(1000);

  Serial.println(F("-> 0 grados"));
  moverAGrados(0.0);
  delay(2000);
}
```

Código 5.2. Motor paso a paso con driver `A4988` o `DRV8825`, con búsqueda de origen.

ANTES DE SEGUIR, VERIFICA

- ☐ Calculaste el par de cada articulación con el brazo horizontal y la carga útil incluida.
- ☐ Aplicaste un factor de seguridad de al menos 2 antes de elegir el modelo.
- ☐ Los servos tienen fuente propia de 5–6 V, con capacitor de desacople y masa común con el Arduino.
- ☐ Probaste cada articulación por separado antes de montarla.
- ☐ La Tabla 5.3 está completa, con los cuatro datos medidos de cada articulación.

RESUMEN DEL CAPÍTULO

- Una articulación se elige por par, recorrido, resolución, repetibilidad y retención; el par es el que decide.
- El servomotor es de posición absoluta y se controla con el ancho de un pulso: usa `writeMicroseconds()`, diez veces más fino que `write()`.
- El motor paso a paso es de lazo abierto: necesita driver, ajuste de corriente y búsqueda de origen, y pierde pasos sin avisar.
- Para brazos de 2 a 4 GDL los servos son la elección por defecto; los paso a paso ganan en ejes largos o de giro continuo.

- El par se dimensiona sumando masa por distancia con el brazo horizontal, y se multiplica por un factor de seguridad de 2.
- El hombro necesita del orden de diez veces el par de la muñeca: los motores grandes van cerca de la base.
- Los servos jamás se alimentan desde el Arduino, y toda articulación se prueba sola antes de montarse.

6 Control por serial con Python y Arduino

EN ESTE CAPÍTULO APRENDERÁS

- Decidir qué parte del trabajo hace el computador y qué parte el microcontrolador.
- Entender cómo funciona una comunicación serial y qué la rompe.
- Diseñar un protocolo de comandos propio, legible y verificable.
- Escribir el firmware que traduce ángulos a pulsos, respeta los límites e interpola el movimiento.
- Controlar el brazo desde Python escribiendo coordenadas, no ángulos.

6.1 Repartir el trabajo

Tienes dos computadores: uno de escritorio con Python y un microcontrolador con el brazo colgando. La primera decisión de este capítulo es qué hace cada uno.

El reparto que usa este libro —y casi todos los brazos de escritorio— es:

El anfitrión (Python)
: Resuelve la cinemática inversa, planifica la trayectoria, decide entre codo arriba y codo abajo, detecta puntos inalcanzables y guarda registros. Piensa en milímetros.

El firmware (Arduino)
: Recibe ángulos, los traduce a pulsos, aplica los límites mecánicos e interpola el movimiento. Piensa en microsegundos.

La Figura 6.1 resume el recorrido de una orden, desde las coordenadas en Python hasta el movimiento de los servos.

Las razones son concretas. Un Arduino UNO tiene 2 KB de memoria y no tiene unidad de punto flotante: cada arcocoseno le cuesta cientos de microsegundos, y cada cambio en la cinemática obliga a recompilar y recargar. En Python la misma cinemática se prueba, se grafica y se corrige en segundos, sin hardware.

NOTA

El reparto contrario —toda la cinemática dentro del Arduino— también es válido y a veces necesario: un brazo que debe funcionar sin computador conectado no tiene alternativa. Es más trabajo, ocupa más memoria y se depura peor, pero el protocolo de este capítulo sigue sirviendo: cambia lo que hay detrás de cada comando, no los comandos.

6.2 Cómo funciona el puerto serial

La comunicación serial manda los bits de a uno por un solo cable, en orden. El periférico que lo hace se llama `UART`, y necesita dos líneas: `TX` para transmitir y `RX` para recibir, cruzadas entre los dos extremos —el `TX` de uno va al `RX` del otro—. Cuando conectas el Arduino por USB, ese cruce ya está resuelto dentro de la placa.

Hay cuatro hechos que explican casi todos los problemas de este capítulo:

1. **Los dos extremos deben usar la misma velocidad.** Si el firmware abre a 115 200 baudios y Python a 9 600, no aparece ningún error: llegan caracteres basura.
2. **Abrir el puerto reinicia la placa.** El Arduino UNO se reinicia cuando el anfitrión abre el puerto. Por eso el firmware imprime `LISTO` al arrancar y Python espera ese mensaje antes de hablar.
3. **El buffer de entrada del UNO son 64 bytes.** Si el anfitrión dispara comandos sin esperar respuesta, se pierden caracteres a mitad de camino y el firmware recibe órdenes truncadas.
4. **Solo un programa puede tener el puerto abierto.** Si el Monitor Serie del Arduino IDE está abierto, Python no va a poder conectarse. Es el error más frecuente de todo el capítulo.

6.3 Diseñar el protocolo

Un **protocolo** es el acuerdo sobre qué mensajes existen y qué significa cada uno. Lo defines tú, y conviene definirlo con cinco reglas:

1. **Texto, no binario.** Un protocolo de texto se lee y se prueba desde el Monitor Serie, escribiendo a mano, sin ningún programa. Esa sola ventaja compensa de sobra lo que ocupa de más.
2. **Una línea, un comando**, terminada en salto de línea. El terminador es lo que le dice al firmware que el mensaje está completo.
3. **Toda orden se responde.** El firmware contesta `OK` o `ERR`, y el anfitrión no envía la siguiente hasta recibirla. Eso es control de flujo, y es lo que impide desbordar los 64 bytes del buffer.
4. **Los errores se dicen, no se ignoran.** Un comando mal formado responde `ERR` con el motivo.
5. **El firmware no confía en el anfitrión.** Aunque Python ya verificó el alcance, el firmware recorta cada ángulo a sus límites mecánicos. Es la última barrera antes de que un servo choque contra el chasis.

CONSEJO PRÁCTICO

Fíjate en la utilidad de `GET`: devuelve el ángulo que quedó, no el que se pidió. Si un límite mecánico recortó el movimiento, el anfitrión se entera comparando ambos. Un brazo que no puede informar dónde está es un brazo que no puedes depurar.

6.4 La primera prueba: el eco

Antes del brazo, antes de la cinemática, antes de todo: comprobar que los dos extremos se entienden. El Código 6.1 recibe una línea y la devuelve precedida de `OK`.

Tabla 6.1. Protocolo de comandos del brazo. Cada línea termina en salto de línea.

Comando	Respuesta	Qué hace
`PING`	`OK PONG`	Comprueba que el enlace está vivo
`MOVE a0 a1 a2 a3`	`OK`	Lleva las cuatro articulaciones a esos ángulos, en grados del modelo
`GRIP 0` | `GRIP 1`	`OK`	Abre o cierra la pinza
`SPEED n`	`OK`	Milisegundos entre pasos de interpolación: más alto, más lento
`HOME`	`OK`	Lleva el brazo a su postura de reposo
`GET`	`OK POS a0 a1 a2 a3`	Devuelve los ángulos que el brazo tiene realmente
Cualquier otra cosa	`ERR` *motivo*	Comando desconocido o mal formado

```arduino
// Eco por puerto serial: el programa más pequeño que demuestra que Python y
// Arduino se están entendiendo. Recibe una línea terminada en '\n' y la
// devuelve precedida de "OK ". Si esto no funciona, nada de lo que venga
// después va a funcionar (Capítulo 6).

const int LARGO_MAX = 48;    // el buffer serial del UNO son 64 bytes
char linea[LARGO_MAX];
int cuantos = 0;

void setup() {
  Serial.begin(115200);
  // Espera a que el puerto esté listo y anuncia que el firmware arrancó.
  // El anfitrión usa este mensaje para saber que puede empezar a hablar.
  delay(200);
  Serial.println(F("LISTO"));
}

void loop() {
  while (Serial.available() > 0) {
    char c = Serial.read();

    if (c == '\n' || c == '\r') {
      if (cuantos == 0) continue;    // ignora líneas vacías
      linea[cuantos] = '\0';         // cierra la cadena
      Serial.print(F("OK "));
      Serial.println(linea);
      cuantos = 0;
    } else if (cuantos < LARGO_MAX - 1) {
      linea[cuantos++] = c;
    } else {
      // Línea más larga que el buffer: se descarta entera y se avisa.
      cuantos = 0;
      Serial.println(F("ERR linea muy larga"));
    }
  }
}
```

Código 6.1. Eco por puerto serial: la prueba mínima del enlace.

Se prueba sin escribir una línea de Python: carga el sketch, abre el Monitor Serie a 115 200 baudios, escribe cualquier cosa y presiona Enter. Si vuelve precedida de `OK`, el enlace funciona.

Si vuelven caracteres extraños, la velocidad no coincide.

NOTA

El sketch lee carácter por carácter y arma la línea en un arreglo propio en vez de usar objetos `String`. En un microcontrolador con 2 KB de memoria, las cadenas que crecen y se liberan solas terminan fragmentando la memoria y provocando reinicios difíciles de diagnosticar. Un arreglo de tamaño fijo no tiene ese problema.

6.5 Del ángulo del modelo al pulso del servo

Aquí se cierra el asunto que quedó abierto en el Capítulo 2. Tu modelo trabaja en grados con un cero geométrico; el servo trabaja en microsegundos con un cero propio y quizás con el sentido de giro invertido. La traducción tiene una sola línea:

$$\text{pulso} = \text{pulso}_{\text{cero}} + \theta \cdot k \qquad \text{recortado a } [\text{pulso}_{\text{min}},\ \text{pulso}_{\text{max}}] \tag{6.1}$$

donde k son los microsegundos por grado —típicamente $1000/90 = 11.11$— con **signo**: negativo si el servo gira al revés que tu modelo. Los cuatro valores por articulación son los que mediste en la Tabla 5.3 del Capítulo 5.

En el firmware eso se escribe como una tabla, una fila por articulación:

```
// Calibración de cada articulación. Se obtiene midiendo, no calculando:
// el cero del modelo cinemático casi nunca coincide con el centro del servo.
struct Articulacion {
  byte pin;
  int pulsoCero;      // us que corresponden a 0 grados del modelo
  float usPorGrado;   // el signo corrige el sentido de giro
  int pulsoMin;       // tope mecánico seguro
  int pulsoMax;
  float anguloHome;
};

Articulacion junta[N_JUNTAS] = {
  {  9, 1500,  11.11, 700, 2300,   0.0 },  // 0 base
  { 10, 1480, -11.11, 700, 2300,  60.0 },  // 1 hombro
  { 11, 2100,  11.11, 500, 2300, -80.0 },  // 2 codo
  {  6, 1500, -11.11, 700, 2300, -40.0 },  // 3 muñeca
};
```

Código 6.2. Calibración del brazo dentro del firmware: los datos medidos en el Capítulo 5, y nada más.

Cambiar de brazo es cambiar esa tabla. Ninguna otra parte del firmware conoce la mecánica.

ATENCIÓN

Los límites `pulsoMin` y `pulsoMax` no son un detalle de programación: son lo único que impide que un servo empuje un eslabón contra el chasis. Un servo bloqueado consume su corriente máxima, se calienta y termina destruyendo su engranaje plástico en menos de un minuto. Mídelos antes de montar y no los toques después.

6.6 El firmware del brazo

El firmware completo, `cap06-02-firmware-brazo.ino`, está en el repositorio del libro (Anexo C). Aquí interesan tres decisiones que hay dentro de él.

6.6.1 La interpolación

Si le mandas a un servo un pulso nuevo, va hacia allá a su máxima velocidad. Con cuatro servos moviéndose así a la vez, el brazo da un tirón, oscila y suelta lo que lleve en la pinza.

La función `moverA()` evita eso: calcula cuál es la articulación que más tiene que moverse, divide ese recorrido en pasos pequeños y avanza todas las articulaciones un paso a la vez. Como todas usan el mismo número de pasos, **parten y llegan juntas**, que es lo que hace que el movimiento se vea coordinado y no como cuatro motores independientes.

```
int anguloAPulso(int i, float grados) {
  long pulso = junta[i].pulsoCero + (long)(grados * junta[i].usPorGrado);
  return constrain(pulso, junta[i].pulsoMin, junta[i].pulsoMax);
}

// Mueve todas las articulaciones a la vez, en pasos pequeños. Sin esto los
// servos salen disparados a máxima velocidad y el brazo pierde la carga.
void moverA(float destino[N_JUNTAS]) {
  int objetivo[N_JUNTAS];
  int mayorSalto = 0;

  for (int i = 0; i < N_JUNTAS; i++) {
    objetivo[i] = anguloAPulso(i, destino[i]);
    int salto = abs(objetivo[i] - pulsoActual[i]);
    if (salto > mayorSalto) mayorSalto = salto;
  }

  int pasos = mayorSalto / PASO_US;
  if (pasos < 1) pasos = 1;

  int inicio[N_JUNTAS];
  for (int i = 0; i < N_JUNTAS; i++) inicio[i] = pulsoActual[i];

  for (int p = 1; p <= pasos; p++) {
    for (int i = 0; i < N_JUNTAS; i++) {
      long avance = (long)(objetivo[i] - inicio[i]) * p / pasos;
      pulsoActual[i] = inicio[i] + (int)avance;
      servo[i].writeMicroseconds(pulsoActual[i]);
    }
    delay(msPorPaso);
  }

  for (int i = 0; i < N_JUNTAS; i++) {
    pulsoActual[i] = objetivo[i];
    // Se guarda el ángulo que realmente quedó, no el pedido: si el límite
    // mecánico recortó el movimiento, el anfitrión debe enterarse.
    anguloActual[i] = (objetivo[i] - junta[i].pulsoCero) / junta[i].usPorGrado;
  }
}
```

Código 6.3. Traducción a pulsos e interpolación coordinada de las cuatro articulaciones.

NOTA

Esta interpolación es *bloqueante*: mientras dura el movimiento, el firmware no atiende el puerto serial. Aquí no es un problema, porque el anfitrión está esperando la respuesta `OK` de todas formas. Si más adelante necesitas que el brazo reciba órdenes mientras se mueve, o que lea un final de carrera o un botón de paro durante el trayecto, hay que reescribirla sin `delay()`, avanzando un paso cada vez que `millis()` indique que pasó el tiempo. Mientras no lo hagas, el firmware *no* puede detener el brazo a mitad de un movimiento: pon un interruptor en la alimentación de los servos para cortarla a mano.

6.6.2 El ángulo que se guarda

Después de mover, el firmware no guarda el ángulo que le pidieron: recalcula el ángulo a partir del pulso que realmente quedó, después del recorte por límites. Así `GET` dice la verdad aunque la orden haya sido imposible.

6.6.3 Los mensajes en memoria de programa

Todos los textos van dentro de `F()`. Esa macro deja la cadena en la memoria de programa —32 KB— en vez de copiarla a la memoria de variables —2 KB—. En un firmware con veinte mensajes, la diferencia es la mitad de la RAM disponible.

6.7 El anfitrión en Python

Del otro lado, Python necesita una sola biblioteca externa:

```
pip install pyserial
```

La clase `Brazo` —completa en `control_brazo.py`, en el repositorio— encapsula las cuatro cosas que siempre hay que hacer y que siempre se olvidan: encontrar el puerto, esperar el reinicio de la placa, esperar la respuesta de cada comando y convertir un `ERR` del firmware en una excepción de Python en vez de un error silencioso. Ese es su núcleo:

```
        # Abrir el puerto reinicia la placa: hay que esperar a que arranque.
        # El firmware avisa con "LISTO" cuando termina de posicionar los servos.
        self._esperar("LISTO")

    def _esperar(self, texto):
        limite = time.time() + self.tiempo_limite
        while time.time() < limite:
            linea = self.serie.readline().decode("utf-8", "ignore").strip()
            if linea == texto:
                return
        raise ErrorBrazo("el firmware nunca respondio %r" % texto)

    def comando(self, texto):
        """Envia una linea y devuelve la respuesta del firmware."""
        self.serie.reset_input_buffer()
        self.serie.write((texto + "\n").encode("utf-8"))

```

```
        limite = time.time() + self.tiempo_limite
        while time.time() < limite:
            respuesta = self.serie.readline().decode("utf-8", "ignore").strip()
            if respuesta.startswith("ERR"):
                raise ErrorBrazo("%s -> %s" % (texto, respuesta))
            if respuesta.startswith("OK"):
                return respuesta
        raise ErrorBrazo("sin respuesta a %r" % texto)
```

Código 6.4. El corazón del anfitrión: esperar el arranque de la placa y no enviar un comando hasta que el anterior haya sido contestado.

CONSEJO PRÁCTICO

Fíjate en el uso de `with Brazo() as brazo:`. El puerto serial queda cerrado aunque el programa falle a mitad de camino. Sin eso, una excepción deja el puerto tomado y el siguiente intento falla con un mensaje que no tiene nada que ver con la causa real.

6.8 Manos a la obra: el brazo completo

MANOS A LA OBRA

Arma la cadena completa, en este orden y sin saltarte pasos:

1. Carga el eco (Código 6.1) y verifica el enlace desde el Monitor Serie.
2. Carga el firmware completo (`cap06-02-firmware-brazo.ino`, del repositorio) con tu propia tabla de calibración, todavía con los eslabones desmontados.
3. Desde el Monitor Serie, escribe `PING` y confirma que responde `OK PONG`. Después `HOME`, y observa que cada servo va a su postura de reposo.
4. Monta los eslabones y prueba `MOVE 0 60 -80 -40` a mano, desde el Monitor Serie. Corrige los signos de la tabla si alguna articulación se mueve al revés.
5. **Cierra el Monitor Serie** y ejecuta el anfitrión de Python (`control_brazo.py`, del repositorio).
6. Compara lo que pediste con lo que devuelve `GET`. Si alguna articulación quedó recortada, revisa sus límites.

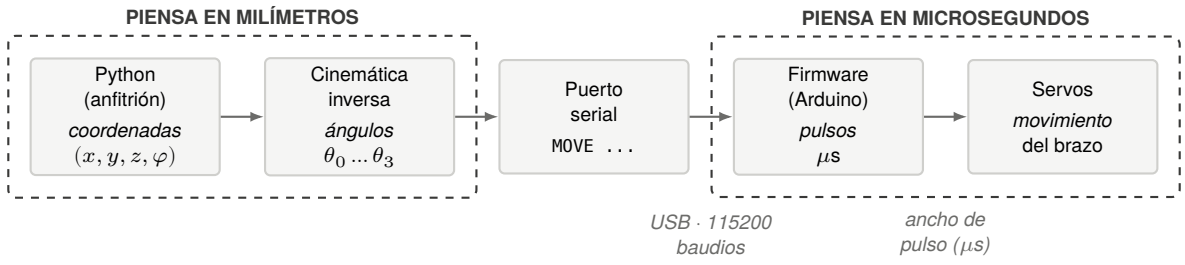


Figura 6.1. La cadena completa de control: el anfitrión resuelve la geometría en milímetros y grados, el firmware traduce a microsegundos y mueve los servos. El puerto serial es la única frontera entre ambos mundos.

6.9 Depuración

Cuando algo no funciona, el orden de revisión ahorra horas:

1. **¿Responde el eco?** Si no, el problema es el enlace: cable, puerto o velocidad. No sigas.
2. **¿Responde `PING` desde el Monitor Serie?** Si sí, el firmware está bien y el problema está en Python.
3. **¿Está el Monitor Serie abierto?** Ciérralo antes de correr Python.
4. **¿Coinciden los baudios** en el firmware, en el Monitor Serie y en Python?
5. **¿Comparten masa** la fuente de los servos y el Arduino? Sin GND común, las señales no tienen referencia y el brazo se mueve al azar.
6. **¿Se reinicia el Arduino al mover?** Es la fuente de los servos: revisa el Capítulo 5.

ANTES DE SEGUIR, VERIFICA

- [ ] El eco responde correctamente desde el Monitor Serie.
- [ ] `PING` devuelve `OK PONG` y `HOME` lleva el brazo a reposo.
- [ ] Tu tabla de calibración está cargada y ninguna articulación se mueve en el sentido contrario al del modelo.
- [ ] El anfitrión de Python conecta, mueve el brazo y cierra el puerto sin errores.
- [ ] `GET` devuelve los mismos ángulos que pediste, sin recortes inesperados.

RESUMEN DEL CAPÍTULO

- El anfitrión piensa en milímetros y resuelve la cinemática; el firmware piensa en microsegundos y mueve los servos.
- Un protocolo de texto, de una línea por comando y con respuesta obligatoria, se depura desde el Monitor Serie y no desborda el buffer.
- El firmware nunca confía en el anfitrión: recorta todo ángulo a los límites mecánicos.
- La calibración es una tabla de cuatro números por articulación, medidos en el Capítulo 5, y es lo único del firmware que depende de tu mecánica.
- La interpolación hace que todas las articulaciones partan y lleguen juntas: sin ella el brazo da tirones y suelta la carga.
- Casi todas las fallas son enlace, baudios, puerto ocupado o masa común; revísalas en ese orden antes de tocar el código.

7 Proyectos y hacia dónde seguir

EN ESTE CAPÍTULO APRENDERÁS

- Especificar tu propio robot articulado con un procedimiento ordenado.
- Elegir un proyecto de 2, 3 o 4 grados de libertad acorde a tu tarea y a tu experiencia.
- Ver funcionando la cadena completa del libro en un ciclo de recoger y soltar.
- Saber qué viene después y qué limitaciones tiene lo que construiste.

7.1 Especificar tu propio robot en seis pasos

Todo lo que hiciste hasta aquí converge en este procedimiento. Cada paso tiene su capítulo, y ninguno se puede saltar sin pagarlo más adelante.

1. **Escribe la tarea en una frase.** Qué objeto, desde dónde, hasta dónde, con qué precisión y con qué peso. Si no cabe en una frase, todavía no está definida.
2. **Cuenta los grados de libertad** que pide esa frase, con la Tabla 2.1. Ni uno más (Capítulo 2).
3. **Fija el espacio de trabajo.** Distancia máxima y mínima, altura, y dónde va la base. Deja la tarea entre el 30 % y el 90 % del alcance (Capítulos 1 y 4).
4. **Verifica la cinemática antes de fabricar.** Pon tus largos en el código del Capítulo 4 y comprueba que todos los puntos de la tarea tienen solución con ángulos que un servo real pueda dar.
5. **Dimensiona los motores** sumando masa por distancia con el brazo horizontal, con factor de seguridad 2 (Capítulo 5).
6. **Define el protocolo** y arma la cadena de control, probando el eco antes que nada (Capítulo 6).

CONSEJO PRÁCTICO

Los pasos 4 y 5 son los que la mayoría se salta, y son justamente los dos que solo cuestan una tarde en el computador. Descubrir con el brazo armado que el punto de destino queda 20 mm fuera de alcance, o que el hombro no levanta la pieza, cuesta el proyecto entero.

7.2 Proyectos de 2 grados de libertad

Son el mejor primer robot articulado: dos servos, poca estructura y una cinemática que puedes verificar a mano.

Torreta orientable (pan-tilt)
: Dos servos en ejes perpendiculares que apuntan una cámara, un puntero o un sensor. Es el único proyecto de esta lista que no necesita cinemática inversa: los dos ángulos *son* la orden. Sirve para entender el hardware y el control serial sin la geometría encima.

Brazo dibujante plano
Dos eslabones sobre una hoja horizontal, con un tercer servo que solo sube y baja el lápiz —y que, como no mueve la punta por el plano, no cuenta como grado de libertad. Usa la cinemática inversa de 2 GDL del Capítulo 4 en su forma más pura.

Trasladador de piezas en un plano
Toma piezas de un punto y las deja en otro, todo a la misma altura, con una ventosa o un electroimán.

> **NOTA**
> El brazo dibujante enseña algo que ningún otro proyecto muestra tan claro: como la orientación no se controla, el lápiz llega inclinado de forma distinta en cada punto de la hoja. Si eso importa —por ejemplo, con un pincel— necesitas el tercer grado de libertad, y ahí se entiende para qué sirve.

7.3 Proyectos de 3 grados de libertad

Brazo de recoger y soltar con ventosa
Base giratoria más hombro y codo. La ventosa apunta siempre hacia abajo por construcción, así que no hace falta articulación de muñeca. Alcanza cualquier (x, y, z) dentro de su espacio de trabajo: es el brazo más simple que resuelve una tarea tridimensional completa.

Brazo dibujante con orientación
Tres articulaciones en un plano, que mantienen la herramienta perpendicular a la superficie en todo el recorrido. Es el caso de 3 GDL plano del Capítulo 4, con el punto de muñeca.

SCARA de escritorio
Dos articulaciones rotacionales de eje vertical más un eje vertical prismático. Muy rígido en altura y muy rápido en el plano; su cinemática inversa es exactamente la de 2 GDL, con la altura resuelta por separado.

7.4 Proyectos de 4 grados de libertad

Brazo de escritorio con pinza
El brazo de referencia de este libro: base, hombro, codo, muñeca y pinza. Controla posición e inclinación de la herramienta, que es lo que hace falta para tomar objetos que no están horizontales o para meterlos en un hueco.

Clasificador por color
El mismo brazo, más un sensor de color sobre la zona de recogida. El anfitrión lee el color, elige el contenedor de destino y ejecuta el ciclo. Es el primer proyecto donde el robot *decide* y no solo repite.

Brazo de servicio de escritorio
El mismo brazo con una lista de posiciones memorizadas, que traslada objetos entre puntos fijos por orden del usuario.

Tabla 7.1. Los proyectos de un vistazo. La dificultad es acumulativa: cada nivel supone resuelto el anterior.

Proyecto	GDL	Actuadores	Lo nuevo que exige
Torreta pan-tilt	2	2 servos pequeños	Nada: control directo
Brazo dibujante plano	2	2 servos medianos	Cinemática inversa de 2 GDL
Trasladador plano	2	2 servos medianos	Ventosa o electroimán
Recoger con ventosa	3	3 servos, uno grande	Espacio tridimensional
Dibujante orientado	3	3 servos medianos	Punto de muñeca
SCARA de escritorio	3	2 servos + 1 paso a paso	Eje prismático y su origen
Brazo con pinza	4	4 servos + pinza	Dimensionamiento serio del hombro
Clasificador por color	4	4 servos + sensor	Decisión en el anfitrión

7.5 La cadena completa: recoger y soltar

Este es el programa que junta los seis capítulos. No contiene un solo ángulo escrito a mano: se le dan coordenadas en milímetros, la cinemática inversa del Capítulo 4 las convierte en ángulos y el anfitrión del Capítulo 6 los envía al firmware.

```
"""Secuencia de recoger y soltar: junta todas las piezas del libro.

Se escriben coordenadas cartesianas en milimetros; la cinematica inversa
del Capitulo 4 las convierte en angulos y el anfitrion del Capitulo 6 los
envia al firmware. En ningun momento se escribe un angulo a mano.

Ejecutalo con el brazo conectado:
    python recoger_y_soltar.py
"""

import math
import os
import sys
import time

# Los modulos de los capitulos anteriores viven en sus propias carpetas.
AQUI = os.path.dirname(os.path.abspath(__file__))
for carpeta in ("cap04-cinematica-inversa", "cap06-03-control-brazo"):
    sys.path.append(os.path.join(AQUI, "..", carpeta))

from cinematica_inversa import ik_4gdl          # noqa: E402
from control_brazo import Brazo, ErrorBrazo     # noqa: E402


def ir_a(brazo, x, y, z, inclinacion_grados=-45.0):
    """Lleva la punta de la herramienta a un punto del espacio."""
    angulos = ik_4gdl(x, y, z, math.radians(inclinacion_grados))
    if angulos is None:
        raise ErrorBrazo("el punto (%.0f, %.0f, %.0f) esta fuera de alcance"
                         % (x, y, z))
```

```
    brazo.mover([math.degrees(a) for a in angulos])


# Puntos de trabajo, en milimetros desde el centro de la base.
# Cambiar la tarea es cambiar esta tabla, no el codigo.
ORIGEN = (170.0, -60.0, 40.0)
DESTINO = (170.0, 60.0, 40.0)
ALTURA_SEGURA = 120.0     # altura a la que el brazo se desplaza sin chocar


def recoger_y_soltar(brazo, origen, destino):
    x1, y1, z1 = origen
    x2, y2, z2 = destino

    ir_a(brazo, x1, y1, ALTURA_SEGURA)   # se acerca por arriba
    brazo.pinza(False)                   # abre antes de bajar
    ir_a(brazo, x1, y1, z1)              # baja sobre la pieza
    brazo.pinza(True)                    # agarra
    time.sleep(0.3)

    ir_a(brazo, x1, y1, ALTURA_SEGURA)   # levanta
    ir_a(brazo, x2, y2, ALTURA_SEGURA)   # traslada en alto
    ir_a(brazo, x2, y2, z2)              # baja
    brazo.pinza(False)                   # suelta
    time.sleep(0.3)
    ir_a(brazo, x2, y2, ALTURA_SEGURA)   # se retira


if __name__ == "__main__":
    with Brazo() as brazo:
        brazo.velocidad(10)
        brazo.home()

        recoger_y_soltar(brazo, ORIGEN, DESTINO)

        brazo.home()
        print("Ciclo terminado.")
```

Código 7.1. Ciclo de recoger y soltar, en coordenadas cartesianas.

Observa la estructura del ciclo, porque se repite en casi cualquier tarea de manipulación: acercarse *por encima* del objeto, bajar recto, agarrar, subir recto, trasladar en alto y volver a bajar. Nunca se va en diagonal desde el origen al destino: así es como se arrastra la pieza o se choca con lo que haya en medio.

NOTA

Cambiar la tarea es cambiar dos líneas: `ORIGEN` y `DESTINO`. Esa es la ventaja de haber separado la geometría del control. Si los ángulos estuvieran escritos a mano, cada cambio de posición obligaría a recalcularlos todos.

7.6 Lo que este brazo todavía no hace

Vale la pena ser explícito sobre los límites de lo que construiste, porque cada uno señala el siguiente paso.

No se mueve en línea recta
: La interpolación del firmware es en el espacio de las articulaciones: los cuatro servos van linealmente de un ángulo a otro, y la punta describe una curva, no un segmento. Para trayectos rectos hay que subdividir el camino *en Python*, resolver la cinemática inversa en cada punto intermedio y enviar una secuencia de comandos `MOVE`. Es el paso siguiente natural, y no necesita hardware nuevo.

No sabe si llegó
: Los servos no informan su posición real. Un encoder en cada eje, o servos con protocolo digital de realimentación, cierran el lazo y permiten detectar que la carga frenó el brazo.

No controla la velocidad ni la aceleración
: El comando `SPEED` fija un ritmo constante. Un perfil trapezoidal —acelerar, mantener, frenar— reduce las oscilaciones y permite mover más rápido con la misma mecánica.

No ve nada
: Una cámara sobre la zona de trabajo, con detección de objetos, convierte al brazo de una máquina que repite en una que responde a lo que hay sobre la mesa.

7.7 Cierre

Si llegaste hasta aquí con un brazo respondiendo a coordenadas, ya no tienes un proyecto: tienes un método. Definiste una tarea, contaste sus grados de libertad, resolviste su geometría, dimensionaste sus motores y montaste una arquitectura de control con un protocolo propio. Ese recorrido es el mismo para un brazo de escritorio de 250 mm y para uno industrial de dos metros; lo que cambia entre ellos son los números, no las decisiones.

Lo más transferible no es ninguna fórmula en particular. Es el hábito de verificar antes de fabricar: la prueba de ida y vuelta de la cinemática, la suma de momentos antes de comprar un servo, el eco antes del protocolo. Cada una de esas comprobaciones cuesta minutos y cada una evita rehacer una pieza, un cableado o un brazo entero.

Ahora te toca decidir qué construir.

ANTES DE SEGUIR, VERIFICA

- [ ] Recorriste los seis pasos de especificación con tu propio robot.
- [ ] Elegiste un proyecto de la Tabla 7.1 acorde a los grados de libertad que necesitas.
- [ ] El ciclo de recoger y soltar funciona con coordenadas, sin ángulos escritos a mano.
- [ ] Sabes cuáles son los límites de tu brazo y cuál sería tu siguiente mejora.

RESUMEN DEL CAPÍTULO

- Especificar un robot son seis pasos: tarea, grados de libertad, espacio de trabajo, verificación cinemática, dimensionamiento y control.
- Los proyectos de 2 GDL enseñan la cinemática con la mínima mecánica; los de 3 resuelven tareas tridimensionales; los de 4 agregan el control de la orientación de la herramienta.
- Un ciclo de manipulación siempre se acerca por arriba, baja recto, agarra, sube recto y traslada en alto.

- Separar la geometría (Python) del control (firmware) hace que cambiar la tarea sea cambiar coordenadas, no ángulos.
- Los límites actuales del brazo —trayectos curvos, sin realimentación, sin perfil de velocidad y sin visión— son la lista de lo que viene después.

A Formulario

Todas las fórmulas del libro, juntas, para tenerlas a mano mientras programas. Cada una indica el capítulo donde se deduce.

A.1 Notación

θ_0	Ángulo de la base, alrededor del eje vertical
$\theta_1, \theta_2, \theta_3$	Hombro, codo y muñeca; **relativos** al eslabón anterior
L_1, L_2, L_3	Largos de los eslabones
d_0	Altura de la base al eje del hombro
r	Distancia horizontal desde el eje de la base hasta la herramienta
φ	Inclinación de la herramienta, medida desde la horizontal, dentro del plano del brazo

A.2 Cinemática directa (Capítulo 3)

Cadena plana de n eslabones, con el ángulo acumulado $\Theta_k = \theta_1 + \cdots + \theta_k$:

$$x = \sum_{k=1}^{n} L_k \cos \Theta_k, \qquad y = \sum_{k=1}^{n} L_k \sin \Theta_k, \qquad \varphi = \Theta_n$$

Brazo espacial de 4 GDL, resuelto primero en su plano y girado después:

$$r = \sum_{k=1}^{3} L_k \cos \Theta_k, \qquad z = d_0 + \sum_{k=1}^{3} L_k \sin \Theta_k$$

$$x = r \cos \theta_0, \qquad y = r \sin \theta_0$$

A.3 Cinemática inversa (Capítulo 4)

Prueba de alcance, obligatoria antes de calcular nada:

$$|L_1 - L_2| \;\le\; \sqrt{x^2 + y^2} \;\le\; L_1 + L_2$$

Dos grados de libertad, con $r^2 = x^2 + y^2$:

$$\cos\theta_2 = \frac{r^2 - L_1^2 - L_2^2}{2L_1L_2} \qquad \theta_2 = \mp \arccos(\cos\theta_2)$$

(el signo negativo es codo arriba; el positivo, codo abajo)

$$\theta_1 = \text{atan}_2(y, x) - \text{atan}_2(L_2 \sin\theta_2,\ L_1 + L_2\cos\theta_2)$$

Tres grados de libertad en el plano, con orientación φ pedida:

$$x_m = x - L_3\cos\varphi, \qquad y_m = y - L_3\sin\varphi$$

Se resuelve el caso de 2 GDL para (x_m, y_m) y después:

$$\theta_3 = \varphi - \theta_1 - \theta_2$$

Cuatro grados de libertad:

$$\theta_0 = \text{atan}_2(y, x), \qquad r = \sqrt{x^2 + y^2}, \qquad z' = z - d_0$$

y se resuelve el caso de 3 GDL para el punto (r, z') con la orientación φ.

A.4 Denavit-Hartenberg (Capítulo 2)

$$A_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad {}^0T_n = A_1A_2\cdots A_n$$

Tabla del brazo de referencia:

i	**Articulación**	θ_i	d_i	a_i	α_i
1	Base	θ_0	d_0	0	90°
2	Hombro	θ_1	0	L_1	0°
3	Codo	θ_2	0	L_2	0°
4	Muñeca	θ_3	0	L_3	0°

A.5 Movilidad de un mecanismo (Capítulo 2)

Criterio de Grübler, con N eslabones incluida la base, J articulaciones y f_i grados de libertad de cada una:

$$M = 6\,(N - 1 - J) + \sum_{i=1}^{J} f_i \qquad \text{(espacial)}$$

$$M = 3\,(N - 1 - J) + \sum_{i=1}^{J} f_i \qquad \text{(plano)}$$

En una cadena abierta con articulaciones de un grado de libertad, M es el número de articulaciones motorizadas.

A.6 Dimensionamiento de actuadores (Capítulo 5)

Par estático en una articulación, con el brazo horizontal, sumando todas las masas que cuelgan más allá de ella:

$$\tau = \sum_i m_i\, d_i \qquad [\text{kg} \cdot \text{cm}], \quad m_i \text{ en kg},\ d_i \text{ en cm}$$

Se elige el motor con $\tau_{\text{motor}} \geq 2\,\tau$. Conversión: 1 kg · cm ≈ 0.098 N · m.

Resolución en la punta de un brazo de alcance R, para un paso angular $\Delta\theta$ en grados:

$$\Delta s = R \cdot \Delta\theta \cdot \frac{\pi}{180}$$

Pasos por grado de un eje con motor paso a paso:

$$\text{pasos por grado} = \frac{\text{pasos por vuelta} \times \text{micropasos} \times \text{reducción}}{360}$$

A.7 Calibración de una articulación (Capítulo 6)

$$\text{pulso} = \text{pulso}_{\text{cero}} + \theta \cdot k, \qquad \text{recortado a } [\text{pulso}_{\text{min}},\ \text{pulso}_{\text{max}}]$$

con $k \approx \pm 11.11\ \mu$s por grado en un servo típico, y el signo negativo cuando el servo gira al revés que el modelo.

B Guía de resolución de problemas

Todas las fallas de esta guía aparecieron antes en su capítulo. Aquí están juntas, ordenadas por síntoma, para revisar rápido sin recorrer el libro.

Tabla B.1. Cinemática (Capítulos 3 y 4).

Síntoma	Causa probable	Solución
La cinemática inversa devuelve `None` en un punto que se ve alcanzable	La prueba de alcance se hace sobre el *punto de muñeca*, no sobre el objetivo	Cambia la orientación φ pedida, o acorta L_3
El brazo apunta al lado opuesto del objetivo	Se usó `atan` en vez de `atan2`	Reemplaza por `atan2(y, x)` en todas partes
La prueba de ida y vuelta no cierra	Un signo cambiado, o ángulos absolutos donde el modelo espera relativos	Comprueba que cada eslabón usa el ángulo *acumulado* Θ_k
El programa cae con error de dominio en `acos`	El coseno salió fuera de $[-1, 1]$ por redondeo	Recorta el valor a $[-1, 1]$ antes de llamar a `acos`
El brazo se da vuelta a mitad de una trayectoria	Se cambió de codo arriba a codo abajo entre dos puntos	Fija la configuración una vez y no la cambies durante el trayecto
El movimiento se vuelve brusco al final del alcance	Estás entrando en la singularidad del brazo estirado	Mantén la tarea bajo el 90 % del alcance máximo

Tabla B.2. Actuadores y alimentación (Capítulo 5).

Síntoma	Causa probable	Solución
El Arduino se reinicia al mover el brazo	Los servos están alimentados desde el Arduino, o la fuente no da la corriente de pico	Fuente propia de 5–6 V, masa común y capacitor $\geq$1000 µF junto a los servos
Un servo zumba y se calienta sin moverse	Está bloqueado contra un tope mecánico	Corrige `pulsoMin` y `pulsoMax` de esa articulación
El brazo cae solo al soltar la carga	El par del servo del hombro es insuficiente	Rehaz el cálculo con factor de seguridad 2; aliviana o acorta el extremo
La punta bambolea unos milímetros	Holgura de los engranajes del servo, entre 1° y 2°	Es el techo del hardware: usa servos de engranaje metálico o agrega reducción
El motor paso a paso se calienta demasiado	Límite de corriente del driver mal ajustado	Ajusta `Vref` según la hoja de datos, con el motor conectado y detenido
El eje paso a paso se va desfasando	Pérdida de pasos por sobrecarga o aceleración excesiva	Baja la velocidad, sube el par o agrega reducción; y busca el origen más seguido

Tabla B.3. Comunicación y control (Capítulo 6).

Síntoma	Causa probable	Solución
Python no puede abrir el puerto	El Monitor Serie del Arduino IDE está abierto	Ciérralo: solo un programa puede tener el puerto a la vez
Llegan caracteres extraños	Los baudios no coinciden entre firmware y anfitrión	Usa 115 200 en los dos extremos
El primer comando nunca recibe respuesta	Abrir el puerto reinició la placa y el anfitrión habló antes de tiempo	Espera el mensaje `LISTO` antes de enviar nada
Se pierden comandos cuando se envían seguidos	Se desbordó el buffer de 64 bytes del Arduino	Espera la respuesta de cada comando antes de enviar el siguiente
El firmware responde `ERR faltan angulos`	El comando `MOVE` llegó incompleto o con separadores distintos	Un solo espacio entre valores, punto decimal, salto de línea al final
`GET` devuelve ángulos distintos a los pedidos	Los límites mecánicos recortaron el movimiento	Revisa `pulsoMin` y `pulsoMax`, o replantea la postura
Una articulación se mueve al revés	El signo de `usPorGrado` de esa articulación está invertido	Cambia el signo en la tabla de calibración, no en la cinemática
El brazo se mueve de forma errática o al azar	Falta masa común entre la fuente de los servos y el Arduino	Une los GND antes de cualquier otra revisión

C Archivos descargables

Todo el código de este libro, las plantillas de cálculo y el libro mismo en PDF están disponibles para descarga libre en el repositorio:

`https://github.com/EduardoFuentesUA/IntroRobotica`

Los de esta guía están en la carpeta `guia-2-robots-articulados/`. Para bajar todo de una vez, usa el botón **Code** y luego **Download ZIP** en la página del repositorio; no necesitas una cuenta de GitHub.

Tabla C.1. Archivos de esta guía en el repositorio.

Carpeta o archivo	Contenido
`Robots-Articulados.pdf`	Este libro
`codigo/cap03-cinematica-directa/`	Cinemática directa por geometría y por Denavit-Hartenberg, con comprobación cruzada (Capítulo 3)
`codigo/cap04-cinematica-inversa/`	Cinemática inversa cerrada de 2, 3 y 4 GDL, con prueba de ida y vuelta (Capítulo 4)
`codigo/cap05-01-servo-limites/`	Prueba de una articulación con servomotor: límites y rampa (Capítulo 5)
`codigo/cap05-02-stepper-paso-dir/`	Motor paso a paso con driver `A4988` y búsqueda de origen (Capítulo 5)
`codigo/cap06-01-eco-serial/`	Eco por puerto serial: prueba mínima del enlace (Capítulo 6)
`codigo/cap06-02-firmware-brazo/`	Firmware del brazo: protocolo, calibración e interpolación (Capítulo 6)
`codigo/cap06-03-control-brazo/`	Anfitrión en Python: conexión, protocolo y órdenes de alto nivel (Capítulo 6)
`codigo/cap07-01-recoger-y-soltar/`	Ciclo completo en coordenadas cartesianas (Capítulo 7)
`plantillas/hoja-dimensionamiento.ods`	Planilla para sumar momentos y elegir los servos (Capítulo 5)
`plantillas/hoja-calibracion.pdf`	Tabla imprimible de calibración por articulación (Capítulos 5 y 6)

NOTA

Los dos programas de cinemática solo necesitan Python y su biblioteca estándar: puedes ejecutarlos y modificarlos sin ningún componente conectado. El anfitrión y el ciclo de recoger y soltar requieren además `pyserial` (`pip install pyserial`) y el brazo armado.

CONSEJO PRÁCTICO

Si adaptas el código a tu propio brazo, cambia únicamente las constantes de la cabecera de cada archivo: los largos d_0, L_1, L_2, L_3 en los programas de cinemática, y la tabla de calibración en el firmware. Todo lo demás funciona igual para cualquier brazo articulado de 2, 3 o 4 grados de libertad.

La planilla `hoja-dimensionamiento.ods` se abre con LibreOffice Calc, Excel o Google Sheets. Viene cargada con el brazo de referencia del Capítulo 5: reemplaza las masas y distancias por las de tu brazo y la planilla recalcula el par de cada articulación, el par con factor de seguridad y si cada servo de la Tabla 5.1 alcanza.

Bibliografía

[1] Allegro MicroSystems. A4988 dmos microstepping driver with translator, datasheet, 2014.

[2] Arduino. Arduino documentation, 2026. Consultado en 2026.

[3] Arduino. Servo library reference, 2026. Consultado en 2026.

[4] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson, 4 edition, 2018.

[5] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[6] Eduardo Fuentes Fierro. Introducción a la robótica, 2026. Universidad Autónoma de Chile. Primer título de esta serie de guías educativas.

[7] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. Versión electrónica disponible en http://modernrobotics.org/.

[8] Pololu Corporation. Stepper motor driver carriers: Current limit setting, 2026. Consultado en 2026.

[9] pySerial. pyserial documentation, 2026. Consultado en 2026.

[10] Python Software Foundation. The python standard library: math, 2026. Consultado en 2026.

[11] Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2 edition, 2020.

[12] Texas Instruments. Drv8825 stepper motor controller ic, datasheet, 2014.